

DUY TAN UNIVERSITY
SCHOOL OF COMPUTER SCIENCE & ARTIFICIAL INTELLIGENT

Non-Fungible Token for Academic Certification

Course Report

Class: Blockchain

Teacher: Professor Doctor Anand Nayyar

Master Student:
Huynh Thi Bao Khuyen

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Professor Dr. Anand Nayyar for teaching the Blockchain course and for providing the knowledge, learning materials, and guidance that supported the development of this project. I would also like to thank Duy Tan University and the School of Computer Science and Artificial Intelligence for providing the necessary learning environment and resources to complete this work. In addition, I would like to thank the authors of the research papers, technical documents, and open-source projects cited throughout this project. Their contributions have provided valuable insights into blockchain technology, smart contracts, NFTs, decentralized storage, and AI-powered systems. This project has provided valuable practical experience and enhanced my understanding of blockchain-based application development and academic degree verification systems.

Huynh Thi Bao Khuyen
Duy Tan University
June 2026

TABLE OF CONTENTS

Acknowledgement	1
1 Introduction	4
1.1 Background	4
1.2 Problem Statement	5
1.3 Objectives	5
1.4 Scope of the Project	6
1.5 Research Methodology	7
2 Literature review & Theoretical background	7
2.1 Blockchain Fundamentals	7
2.1.1 Decentralization	8
2.1.2 Blocks	8
2.1.3 Hashing	9
2.1.4 Consensus	10
2.2 Smart Contracts	12
2.3 Solidity	13
2.3.1 Global Variables and msg.sender	13
2.3.2 Mappings for Data Storage	13
2.3.3 State Variables and Blockchain Storage	14
2.4 Non-Fungible Token (NFT)	14
2.4.1 The ERC-721 Token Standard & Soulbound (SBT) Modifications	14
2.4.2 Token Metadata, IPFS Schemas, and Dynamic Mapping States	16
2.4.3 Dual-Key Cryptographic Ownership & Hash Verification	16
2.5 Academic Certificate Systems	17
2.5.1 Traditional Academic Verification Process	17
2.5.2 Blockchain-based Verification Architectures	18
2.6 Existing Research Papers	19
2.6.1 Literature Review Matrix	19
2.6.2 Identified Research Gaps and Proposed Technical Solutions	21
3 System analysis & Design	22
3.1 Proposed System	23
3.1.1 Core Engineering Components	24
3.1.2 Addressing Existing Limitations	24
3.1.3 Certificate Verification Workflow	24
3.2 Architecture Diagram	26
3.3 Use Case Diagram	27
3.4 Sequence Diagram	29
3.5 Activity Diagram	30
3.6 Database Diagram	30
3.7 Smart Contract Design	32
3.7.1 Core Functions	33
3.7.2 Soulbound Enforcement	33

4	Implementation	33
4.1	Development Environment	33
4.2	Smart Contract Implementation	35
4.3	Front-end Implementation	37
4.3.1	Admin Console - Create Student Account	37
4.3.2	Student onboard email	38
4.3.3	Student Login	38
4.3.4	Student submits personal wallet address	39
4.3.5	Admin Console - Check for graduation	39
4.3.6	Admin Console - Mint graduate certificate	39
4.3.7	Certificate Search	41
4.4	Back-end API	43
4.5	IPFS Integration	44
4.6	Wallet Integration	45
4.7	Smart Contract Security Analysis	46
4.7.1	Unauthorized Minting Prevention	46
4.7.2	Duplicate Certificate Prevention	48
4.7.3	Soulbound Enforcement	50
4.7.4	Hash Tampering Protection	52
4.7.5	Wallet Ownership Security	55
4.8	Deployment Process	58
4.8.1	Hardhat Local Deployment	58
4.8.2	Smart Contract Compilation	59
4.8.3	Smart Contract Ledger Deployment	60
4.8.4	Environment Syncing and System State Configuration	61
4.8.5	Frontend side for user interface	62
5	AI-Assisted NFT Certificate Verification	63
5.1	AI Assistant Concept	63
5.2	AI Technology Stack	63
5.3	System Architecture and Workflow	64
5.3.1	Step 1 – Certificate Verification	65
5.3.2	Step 2 – User Question Submission	65
5.3.3	Step 3 – Context Construction	67
5.3.4	Step 4 – AI Processing	67
5.3.5	Step 5 – Response Streaming	68
5.4	AI Service Implementation	68
5.5	Prompt Engineering and Hallucination Mitigation	69
6	Results & Conclusion	70
6.1	Project Results	70
6.2	Project Advantages	71
6.3	Project Limitations	72
6.4	Future Work	73
6.4.1	Blockchain-Based Academic Transcript Verification	73
6.4.2	Institution-Specific Large Language Model Development	74
6.5	Source Code Repository	75
6.6	Conclusion	75

1 INTRODUCTION

1.1 Background

Verifying school certificates is still something every employer and university has to do. When people apply for jobs or higher education, someone has to check if their degrees are actually real or just faked. Right now, most places handle this by using centralized databases, sending manual emails, or waiting on paperwork. It is a slow setup that often takes days just to verify a single document.

These traditional habits are common everywhere, but they cause a lot of bottleneck issues, especially in places like Vietnam. For example, if you need to notarize a college degree or a transcript here, you usually have to take a photocopy down to a local government office or a notary place just to prove it matches the original. It takes time. The whole process gets dragged out because office staff are buried in paperwork, rules are too strict, or the schools themselves cannot easily dig up old student files. On top of that, relying on one central database is risky. If a university's server goes down, gets corrupted, or loses old records over time, proving your degree is real becomes almost impossible. This makes the whole setup an easy target for fraud.

Looking closely at traditional verification, the main weaknesses usually come down to three basic problems:

- **Single Points of Failure:** Putting everything on a central university server is a big risk. Servers crash, get hit by cyberattacks, or experience data loss. If a database gets hacked and there is no separate backup system, important student records can be wiped out permanently. There is also the risk of internal staff altering data maliciously.
- **High Delays and Inefficiency:** Checking a degree gets even slower when it involves different countries. HR managers often have to email foreign schools back and forth. Because you have to deal with international privacy laws, extra fees, and slow administrative steps, the process takes forever. This leaves job applicants stuck waiting for weeks just to get approved.
- **Blind Trust in System Admins:** Centralized systems force you to trust whoever runs the database. If an insider with system access wants to be dishonest, they can easily change grades or slip in a fake degree entry. Since outside employers can only see the final screen output, catching this kind of internal tampering is nearly impossible.

To fix these exact weak spots, blockchain technology offers a much better, decentralized way to handle data. Think of a blockchain as a shared digital ledger. Once a record is saved on it, nobody can change it unless the whole network agrees. That specific feature is why it works so well for keeping data secure and completely transparent.

Inside the blockchain ecosystem, Non-Fungible Tokens (NFTs) can be used to handle unique digital ownership. When applied to education, an NFT acts like a digital badge issued directly from a university to a graduate's wallet. Unlike a plain PDF file, an NFT-based certificate uses cryptographic signatures to prove exactly who owns it and where it came from.

The actual workflow connects the digital certificate to the official school record using a hashing method called SHA-256. The system runs the original certificate file through this tool

to generate a unique digital code—a hash—and uploads that code to the blockchain. Later, if an employer wants to check the file, they just upload it to calculate the code again. If the new code matches the original one stored by the university, the degree is instantly verified. If anyone changes even a single letter on the document, the hash changes completely, and the system flags it as a fake right away.

In short, bringing blockchain and NFTs into certificate management fixes the structural flaws of old verification setups. It makes the whole process faster, safer, and completely open, meaning we no longer have to blindly trust a centralized office.

1.2 Problem Statement

Currently, while schools widely use digital databases to manage academic records, existing certificate verification systems still rely heavily on centralized agencies and manual authentication processes. These traditional methods present several significant challenges, including vulnerabilities to certificate forgery, inefficient verification processes, risks of manipulation of centralized data, and limited transparency in authentication procedures. Furthermore, existing systems often struggle to support the rapid and reliable verification of student credentials when studying abroad, causing delays for employers, universities, and organizations requiring verification of overseas academic qualifications. Reliance on centralized databases also increases the risk of data loss, cyberattacks, and internal manipulation. Furthermore, traditional digital certificates cannot effectively provide immutable ownership verification or ensure document integrity in a decentralized environment. As a result, organizations may struggle to establish trust in the authenticity of submitted qualifications.

Thus, a secure, decentralized, transparent, and tamper-proof academic certificate verification system is needed, capable of guaranteeing both document authenticity and ownership verification. Blockchain technology integrated with NFT-based digital certificates offers a potential solution to address these challenges and enhance the reliability of academic qualification verification systems.

1.3 Objectives

The main goal of this project is to build a reliable academic certificate verification platform using blockchain and NFT technology. By doing this, we aim to make checking college degrees much safer, faster, and completely transparent. To achieve this, the project focuses on several specific goals:

- Issue unique, non-transferable NFTs to graduates that serve as permanent digital certificates.
- Build a secure checking tool that uses cryptographic hashing to instantly prove whether a certificate is authentic or has been altered.
- Use the permanent nature of blockchain records to stop degree fraud, text forgery, and unauthorized changes.
- Cut down our reliance on slow, manual paperwork and centralized databases by setting up a decentralized system.

- Make the verification process simple and clear for schools, companies, and any outside organizations checking credentials.
- Deploy an AI-powered assistant to guide users through the platform, explain verification reports, and make the system easier to interact with.
- Test and analyze how well this NFT-based platform holds up in real-world academic scenarios regarding its safety, speed, and overall practicality.

1.4 Scope of the Project

This project focuses entirely on building a blockchain-based platform that uses NFTs to issue and verify school certificates. To keep the project manageable and focused, the scope is strictly limited to the following areas:

1. **Target Users:** The system is built specifically for educational institutions, such as colleges and universities, that need a secure way to hand out academic degrees and professional certificates.
2. **Student Benefits:** Graduates and current students receive secure, non-transferable ownership of their achievements through unique NFT-based credentials.
3. **Third-Party Verification:** Employers, companies, and external organizations can use the platform's blockchain tools to check the authenticity of any submitted certificate instantly.
4. **Core Infrastructure:** We use blockchain technology to keep verification records permanent, alongside Solidity smart contracts to automate how certificates are minted and checked.
5. **Data Integrity:** The system relies on SHA-256 cryptographic hashing. This ensures that if anyone alters a document after it is issued, the system catches the modification immediately.
6. **Smart Contract Testing:** The scope covers writing, deploying, and testing the actual NFT smart contracts responsible for certificate minting and tracking ownership.
7. **Administrative Exclusions:** This platform only handles certificate verification. It deliberately leaves out broader university management features like student enrollment, grading rubrics, tuition tracking, or general school administration systems.
8. **Simplified Web Features:** Standard website elements, like public user registration and login pages, are skipped to keep the focus entirely on the NFT authentication logic. Student accounts are generated directly by the system administrator instead.
9. **Restricted AI Integration:** We integrated an AI agent into the platform, but its role is strictly limited to helping users read and understand their verification reports. It does not handle any core system logic.

1.5 Research Methodology

This project uses a hands-on research and development approach. We combined theoretical research with system design and practical software implementation to build the platform.

The work was divided into the following key phases:

- **Literature Review:** We started by studying academic papers, technical journals, and documentation on blockchain, smart contracts, and cryptographic hashing. This helped us understand the limitations of current setups and identify new technological options.
- **Requirement Analysis:** Next, we identified the functional and non-functional requirements of the system based on the real-world flaws found in traditional verification methods. This included analyzing user roles, security needs, and backend workflows to shape our system architecture.
- **System Design:** We mapped out the entire architecture for the NFT platform. This phase involved structuring the blockchain integration, planning the database schema, designing user interfaces, and mapping out the step-by-step verification flows.
- **Smart Contract Development:** We wrote and deployed the actual smart contracts responsible for handling the core logic of the system, including minting the certificate NFTs, assigning ownership, and managing the verification rules on-chain.
- **Hashing Implementation:** We integrated the SHA-256 cryptographic hashing algorithm into the backend. This allows the system to generate a unique digital footprint for every certificate file, which acts as the main line of defense against document tampering.
- **System Validation:** This phase focused on verifying the core functions of the deployed smart contracts and the web interface. Validation involved running sample certificate files through the SHA-256 hashing pipeline and ensuring that the generated hashes successfully matched the records on the blockchain ledger.
- **Deployment Review:** The final stage involved analyzing the system's behavior during execution. This review confirmed that the platform successfully automates certificate verification and flags modified documents instantly, demonstrating a practical alternative to slow, centralized methods.

2 LITERATURE REVIEW & THEORETICAL BACKGROUND

2.1 Blockchain Fundamentals

Instead of keeping records on one central machine, blockchain splits that data up across a whole network of computers using distributed ledger technology. Since everyone on the network holds their own exact copy of this ledger, going back and secretly changing or faking a record is incredibly hard without getting the whole network to agree on the change. This unique focus on security explains why blockchain keeps getting picked up for real-world setups in banking, tracking supply chains, digital identity, and even IoT tech[1].

You can essentially picture a blockchain as a shared public record books. Transactions get grouped together into individual blocks, and this chain just keeps growing longer as new blocks get tacked onto the end of it over time. To make sure everything stays accurate and nobody can tamper with the data, the network relies on a mix of asymmetric cryptography and distributed

consensus algorithms. When people talk about blockchain's main strengths, they usually point to things like decentralization, permanent records, user anonymity, and audit trails. These exact features are what help companies cut down on extra costs and make verification processes run a whole lot faster.

2.1.1 Decentralization

Most traditional computer setups use an old-school client-server model. In that setup, one company or database manager holds all the keys to the data. That administrator or any user with special rights can easily hop into the database and change things. This setup opens the door to major risks like a single point of failure (SPoF)[2] or internal data fraud. Decentralized networks fix this vulnerability by switching over to a Peer-to-Peer (P2P) setup instead. In a P2P layout, computers talk straight to each other instead of routing everything through a central server[3]. Every single computer acts as both a client and a server at the same time, saving its own live copy of the entire ledger. Because of this layout, no single user can log in and rewrite history from their own computer. If you want to change the system state or add new entries, a strict majority of the network has to approve it first. We will break down exactly how this voting process works in Section 2.1.4.

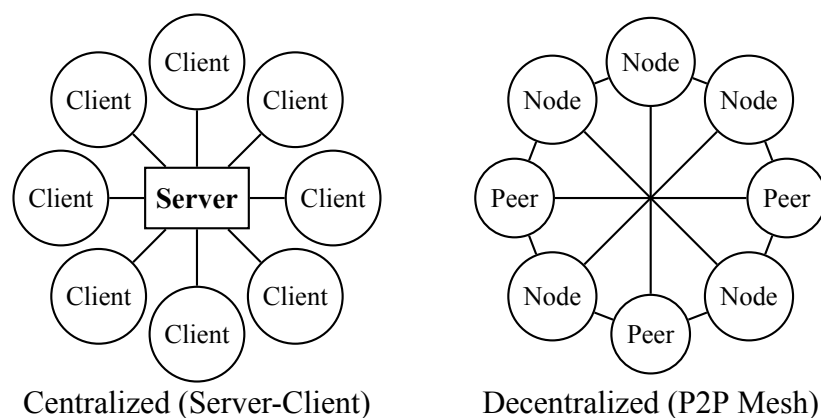


Figure 1: Centralized (Server-Client) and Decentralized (Peer-to-Peer) network

2.1.2 Blocks

A block is really just a data packet used to gather a bunch of transactions or updates together at once. Instead of updating a database line by line, the system bundles records chronologically to make verifying them much easier. Every single block comes with two main pieces: a body and a header. The block body is where the actual transactions sit, along with a counter telling you how many are inside. The total amount of data you can cram into a single block depends entirely on the block size limit and how large each individual transaction happens to be[1]. Over in the block header, you will find these specific tracking details:

- Block version: A basic marker showing which exact set of software validation rules the network needs to follow for this block[1].
- Merkle tree root hash: This is a single cryptographic code that represents the combined fingerprint of every single transaction packed inside that block[1].

- **Timestamp:** Shows the exact moment the block was created, recorded in seconds since the standard start date of January 1, 1970[1].
- **nBits:** A target number that sets how hard it is to calculate a valid hash for the block[1].
- **Nonce:** A small 4-byte counter variable. It starts off at 0 and ticks up by one every single time the system runs a new hash calculation (more on this in Section III)[1].
- **Previous block hash:** A 256-bit cryptographic signature that points straight back to the block right before it, which is what actually locks the chain together[1].

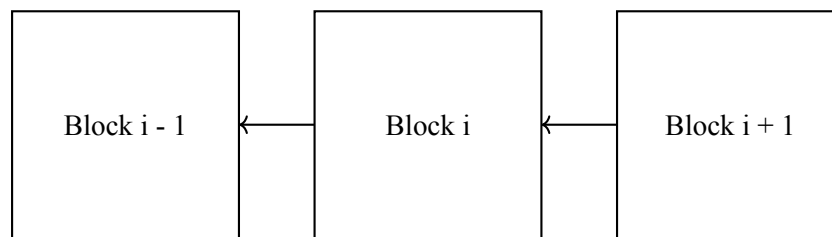


Figure 2: Blockchain architecture

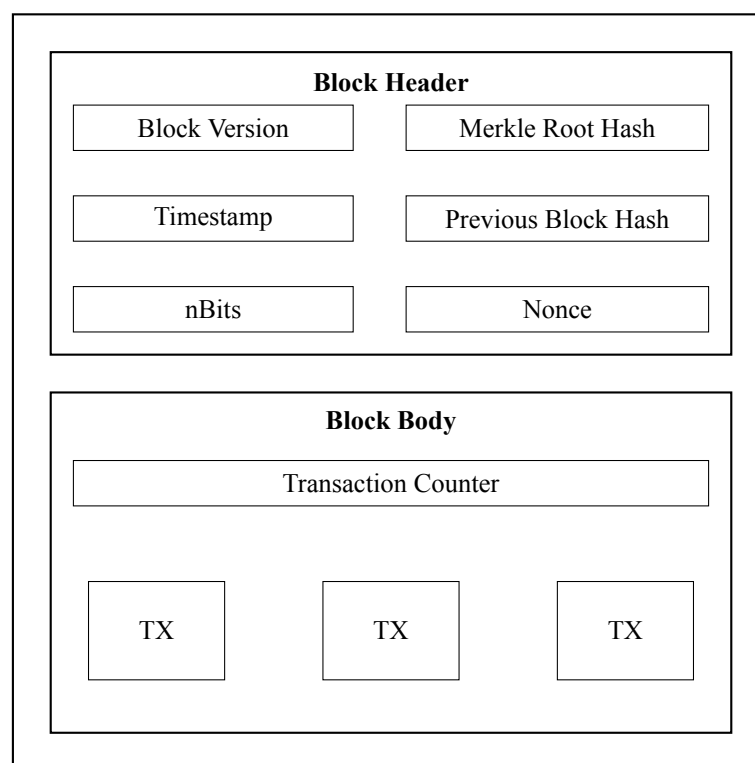


Figure 3: A block properties

2.1.3 Hashing

Hashing is just a method where you pass any kind of input data through a math formula to spit out a fixed-length text string[4]. The moment you run plain text or a file through a hash function, you get a completely unique digital code. Bitcoin and a lot of other big blockchain systems use

an algorithm called SHA-256[5]. It does not matter how big or small your input file is; SHA-256 takes that data and turns it into a standard 256-bit code. The math behind it is incredibly intense, making it practically impossible for regular computers to reverse the hash and figure out what the original file looked like. That said, down the road, breakthroughs in quantum computing might end up threatening the security of these current hashing styles.

2.1.4 Consensus

A consensus mechanism is basically a group protocol used to get an entire network of separate, untrusted computers to agree on one single version of the truth. It completely solves tricky issues like "double-spending" or entering duplicate data entries without needing a boss or a central company to watch over everyone[6].

While developers have come up with several ways to handle consensus—like Proof-of-Work (PoW), Proof-of-Concept (PoC), Proof-of-Stake (PoS), and various mashup models[7]—the two setups you see used the most are PoW and PoS.

- **Proof-of-Work:** Network miners grab pending transactions out of a temporary memory pool (mempool) and piece together candidate blocks. Every candidate block is essentially a draft of what could become the next link in the chain. Miners then run a loop where they change a number called a nonce and run the block header through SHA-256 over and over again. They keep doing this until they get a final hash value that drops below the network's current difficulty target[8]. The moment a miner hits a winning hash, they blast the block out to the rest of the network. Full nodes then double-check the math on their own, and if it looks good, they add it to their personal copy of the chain.

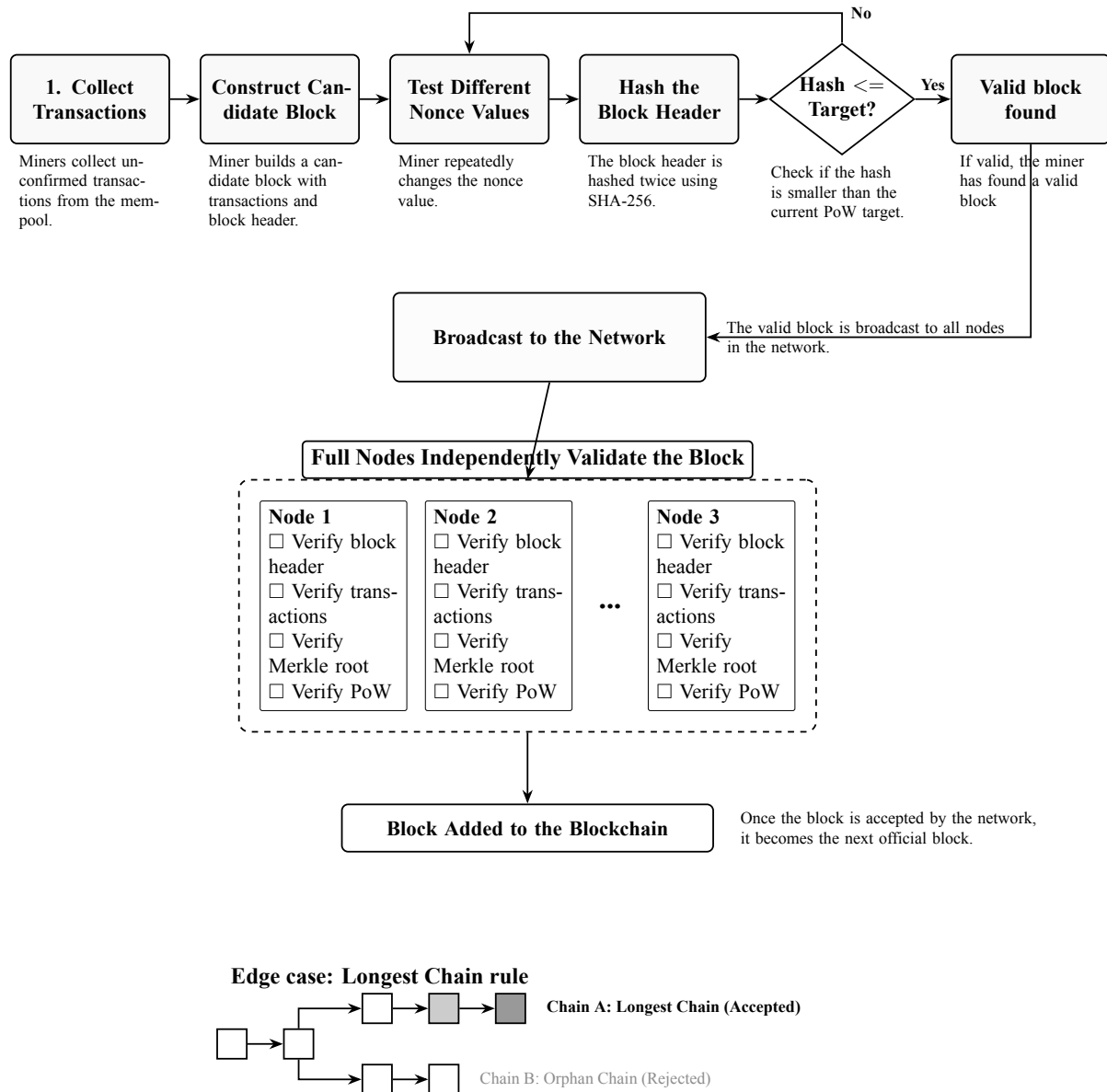


Figure 4: Proof-of-Work workflow

- **Proof-of-Stake:** This setup was built specifically to get around the massive electricity bills and heavy computing power that mining demands. Instead of having computers race each other to solve math puzzles, PoS handpicks block creators (called validators). The system chooses them based on things like how many network coins they have locked up as a deposit, mixed with a bit of random luck[9]. The chosen validator compiles the next block on their own. From there, other network validators check the work, and if they agree it is valid, the block gets officially stamped into the blockchain ledger[7][9].

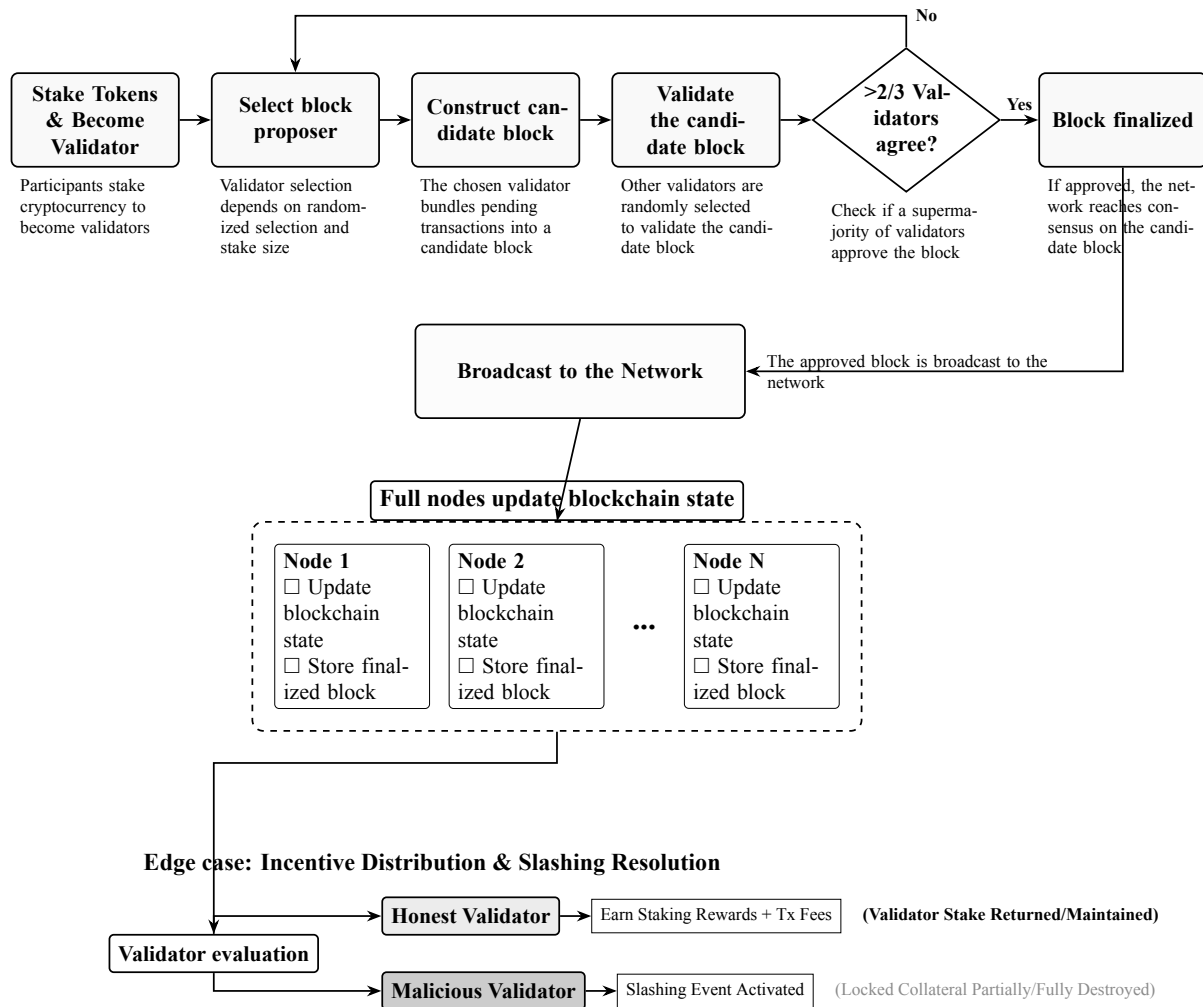


Figure 5: Proof-of-Stake workflow

2.2 Smart Contracts

Moving past the basic building blocks of blockchain—like blocks, hashing, and consensus—we need to look at smart contracts. These programs take blockchain from a simple database and turn it into a programmable machine that can run actual code.

The idea of a "Smart Contract" goes back to Nick Szabo, who famously compared them to a vending machine[10]. In the blockchain world, a smart contract is just code that runs directly on the network. Historically, once you deployed a smart contract, it was impossible to change because of how blockchain works. This caused massive problems during the famous 2016 DAO hack [11]. Because the code was locked forever, developers could not patch the security bug to stop the thief. This disaster actually split Ethereum into two separate chains: Ethereum Classic (ETC) and Ethereum (ETH). The ETC side stuck to the "Code is Law" rule, leaving the stolen history alone and staying with Proof-of-Work. Meanwhile, the main ETH chain chose to do a hard fork to force the stolen money back.

Today, contract code is still technically permanent once deployed on-chain. However, developers now bypass this issue by using a "Proxy Contract" setup. If you find a bug in your original contract (Contract A), you deploy a fixed version (Contract B) and tell the Proxy con-

tract to route all user traffic to the new version instead. It is a bit like updating software without changing the user's link. Also, while ETC still uses mining, ETH eventually switched over to a Proof-of-Stake system in 2022. To prevent another nightmare like the DAO hack, deploying contracts today usually requires safety features like upgradeable proxies, emergency kill-switches, and professional code audits.

Vitalik Buterin described smart contracts as cryptographic "boxes" that hold value and only open up when specific rules are met[12]. They automatically handle complex rules, update system data, and move digital assets around whenever predefined conditions are satisfied[4]. For a school certificate platform, smart contracts act as the backend logic. They control exactly how a degree is generated, tie it to a student's public wallet address, handle global verification checks, or revoke a certificate if an administrative error happens. Most of the time, these contracts are written in a programming language called Solidity[13].

2.3 Solidity

Solidity is the main programming language used to write smart contracts for the Ethereum network. It is a curly-bracket language built specifically to run on the Ethereum Virtual Machine (EVM)[14]. For this certificate project, the code relies heavily on three core Solidity features to handle the logic behind issuing and checking NFTs:

2.3.1 Global Variables and `msg.sender`

Stopping unauthorized users from minting fake certificates is the most critical security rule of this platform. To enforce this, Solidity provides a global variable called `msg.sender`, which checks the exact blockchain address signing and sending the current transaction[14]. By pairing this variable with a basic conditional check, the contract can lock down sensitive functions:

```
modifier onlyAuthorizedAccess() {  
    require(msg.sender == adminAddress, "Error: Unauthorized Caller");  
    _;  
}
```

With this modifier in place, if anyone besides the official university administrator tries to call the `mint()` function, the transaction will instantly fail, stop execution, and cancel any changes.

2.3.2 Mappings for Data Storage

Old-school verification systems use standard SQL databases to search through student records. Solidity replaces that setup with a built-in key-value data structure called a mapping[14]. To keep track of the issued academic NFTs, the contract uses hash-based mapping tables to organize ownership and metadata:

```
mapping(uint256 => address) private _tokenOwners;  
mapping(uint256 => string) private _tokenURIs;
```

- `_tokenOwners`: This connects a specific Certificate Token ID directly to a student's public wallet address. It allows for an incredibly fast $O(1)$ search look-up, meaning anyone can instantly verify who owns a degree without searching through the entire blockchain history.

- **_tokenURIs:** This links that same Token ID to an immutable IPFS Content Identifier (CID) hash string. This string points to the student's actual degree details, such as their name, major, and graduation date.

2.3.3 State Variables and Blockchain Storage

Any variable declared globally at the top of a Solidity contract gets saved directly into persistent blockchain storage (`storage`)[14]. When the university mints an NFT and writes a student's certificate info to this storage space, that data becomes a permanent part of the blockchain history. It cannot be edited, deleted by a corrupt IT worker, or manipulated by a hacker, which guarantees complete and permanent transparency for the degrees.

2.4 Non-Fungible Token (NFT)

To get a clear picture of what a Non-Fungible Token actually means, it is easiest to take the name apart and look at the three ideas behind it:

- **Token:** This is basically a digital asset item locked onto a blockchain setup. It acts as an official receipt showing ownership or specific data values on the shared network[8][15].
- **Fungible:** These are standard items where every unit is identical to the next. You can trade one for another easily because they hold the exact same value, just like standard Bitcoin or cash dollars[8][15].
- **Non-Fungible:** This means something is one-of-a-kind. It has unique traits, so you cannot swap it out for a replacement item or clone it directly[16].

When you bundle these parts together, an NFT is just a unique digital item tracked by blockchain code that proves you own a specific file or digital record.

Using smart contracts and basic mapping logic in Solidity[15], our project uses NFTs to turn academic credentials into unique digital assets[17]. With regular crypto coins like Ether, one coin is exactly the same as another. But with certificate NFTs, every single token has its own data fingerprint on the network[18][16]. This setup makes perfect sense for school diplomas. Since an academic achievement belongs to one specific graduate, it should never be mixed up or traded with someone else. A physical degree belongs to one person, and these certificate tokens work the exact same way—pointing directly to a student's file as permanent proof of graduation[19][17].

2.4.1 The ERC-721 Token Standard & Soulbound (SBT) Modifications

To make sure these tokens actually load properly inside standard crypto wallets and public web pages, we build them using established token standards[16]. The standard layout for handling unique assets on EVM networks is the ERC-721 token blueprint[18][16]. Essentially, this ERC-721 standard lays out the exact coding rules a smart contract uses to mint, follow, and keep track of digital items[16][20]:

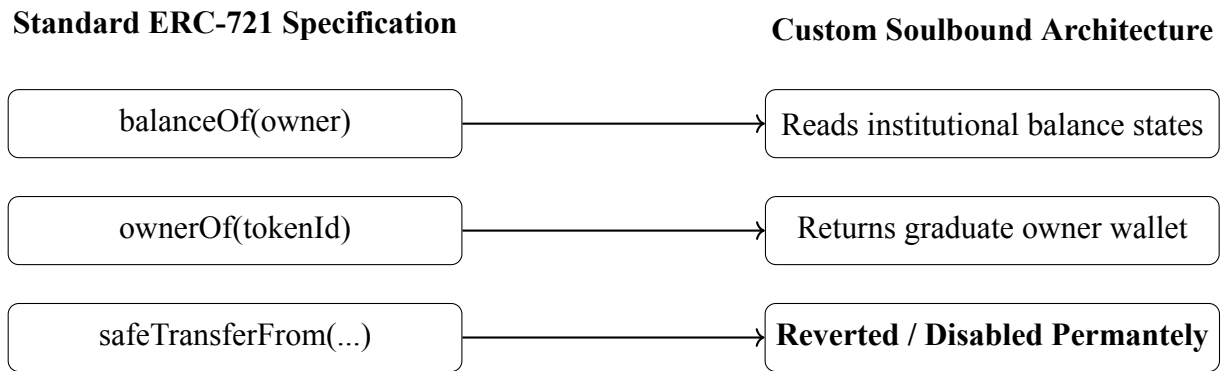


Figure 6: Mapping of ERC-721 Functions to the Custom Soulbound NFT Architecture

- `balanceOf(address owner)`: This function looks up a specific wallet address and tells you exactly how many NFTs are sitting inside it[18].
- `ownerOf(uint256 tokenId)`: You feed this a certificate token ID number, and it tells you the exact wallet address of the owner[18].
- `safeTransferFrom(address from, address to, uint256 tokenId)`: This handles moving an NFT between wallets while making sure the receiving address is capable of accepting ERC-721 assets[18].

The main issue here is that regular ERC-721 tokens are built specifically to be bought and sold. A university degree is a private milestone that nobody should be selling on an open marketplace or giving away to friends. To stop this from happening, our setup alters the standard ERC-721 behavior by stripping out the transfer commands completely. This modifications locks the certificate token to the student's personal wallet forever, turning the asset into a Soulbound Token (SBT)[19].

The Soulbound Constraint Mutation

While regular profile-picture NFTs rely completely on buying and selling tools to trade on marketplaces[18], university certificates need the exact opposite feature: a total block on movement[19][17]. A graduation degree has to stay anchored to the specific student who actually did the schoolwork[19].

To build this restriction, the contract overrides an internal EVM function hook called `_update()` [20]. Any time someone tries to move a token using standard transfer methods like `safeTransferFrom`, the code catches the attempt and checks the wallet destinations[18][20]. If the transaction tries to shift an existing certificate token from one real wallet to another, the script flags it as a violation and stops the execution immediately:

```
if (from != address(0) && to != address(0)) {
    revert("SOULBOUND_ERROR: Certificates cannot be transferred");
}
```

This code block changes how the asset behaves under the hood, turning it into an untradable, non-transferable certificate that stays permanently bound to the student's wallet identity[18][17].

2.4.2 Token Metadata, IPFS Schemas, and Dynamic Mapping States

Trying to save heavy files—like high-quality PDF borders, multi-page layout assets, or student academic transcripts—directly into a blockchain’s main memory is incredibly expensive and unfeasible[15]. To avoid these massive network gas costs, blockchain developers use an off-chain storage method called Token Metadata[18][16]. Instead of forcing the ledger to save massive raw PDF bytes, the smart contract includes a `tokenURI()` function. This function simply outputs a web link pointing to a decentralized JSON file stored off-chain[16][20].

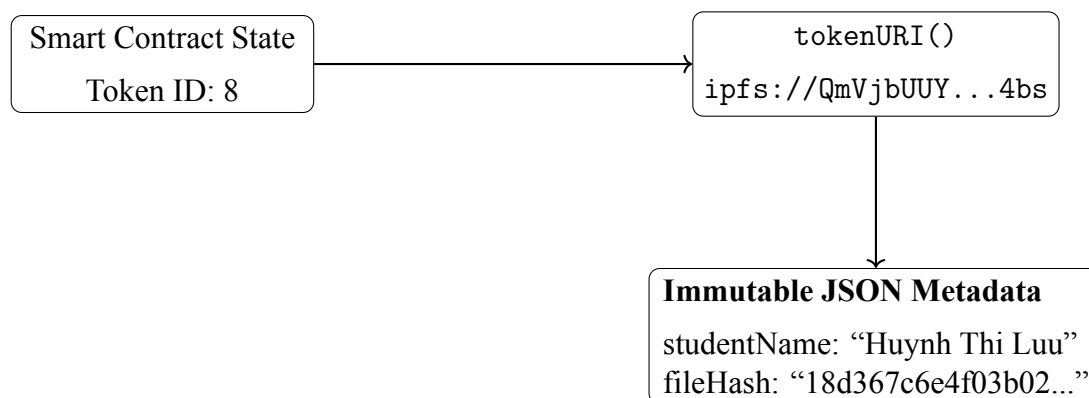


Figure 7: Retrieval of Certificate Metadata via tokenURI

To keep the data formatted correctly so external NFT indexers can read it, the metadata JSON file follows a straightforward schema layout[16]:

- **School Attributes:** A simple data list containing explicit institutional info, like the student’s ID number, their exact degree title, major details, and graduation year.
- **Decentralized File Link:** A fixed IPFS link path (`ipfs://...`) pointing directly to the original certificate PDF document hosted on the InterPlanetary File System[21].

2.4.3 Dual-Key Cryptographic Ownership & Hash Verification

The primary benefit of moving to an NFT setup is the ability to track, prove, and enforce digital records without needing a centralized database room or IT manager[15]. This verification flow relies entirely on public-key cryptography and basic internal smart contract tables[15][22]:

1. **Unique Token ID:** Every single certificate gets stamped with its own unique, sequential ID number when the university mints the block[16].
2. **SHA-256 Fingerprint:** Before saving anything to the ledger, our system runs the raw certificate PDF data through a hashing tool to get a unique 64-character SHA-256 hash string[22]. This string serves as an immutable mathematical fingerprint for that unique file[22][17].
3. **Smart Contract Mappings:** The contract saves and cross-references this information using two internal storage lookup dictionaries[15][20]:

- `mapping(string => CertificateDetails) private _certificatesByHash;`
- `mapping(uint256 => string) private _tokenToHash;`

This sets up a double-sided lookup network. An employer or verification page can check if a degree is authentic simply by running an uploaded PDF file through a web-browser hashing tool[23] and calling the contract with that string[24]. If the blockchain finds a matching, active token linked to that file hash, the degree is instantly verified as authentic[17]. Because modifying these records requires a cryptographic signature from the university's private keys, nobody can overwrite, fake, or tamper with the verification data[15][17]. This gives employers complete proof of custody and institutional validity[16][17].

2.5 Academic Certificate Systems

Integrating decentralized technology into academic administration means confronting a major bottleneck: how current validation frameworks handle data security and day-to-day operations. This section breaks down the actual mechanics of traditional credentialing and pits them against emerging blockchain verification models.

2.5.1 Traditional Academic Verification Process

For decades, academic certificate management has leaned on a mix of paper documents and isolated, centralized databases. When a graduate hands over a degree, third parties usually end up dealing with one of two fundamentally flawed verification loops:

- **Physical Certificates:** Paper credentials rely on physical security—think ink signatures, embossed seals, and complex watermarks to keep forgers at bay. But physical assets are an operational nightmare. Paper rots, gets lost in moves, or vanishes in fires, which forces alumni back to university registrars for manual, slow re-issuance. On top of that, the vetting process happens completely offline. For an employer to verify a degree, they have to physically collect certified hard copies or chase down a university administrator via email or phone. It is slow, analog, and expensive.

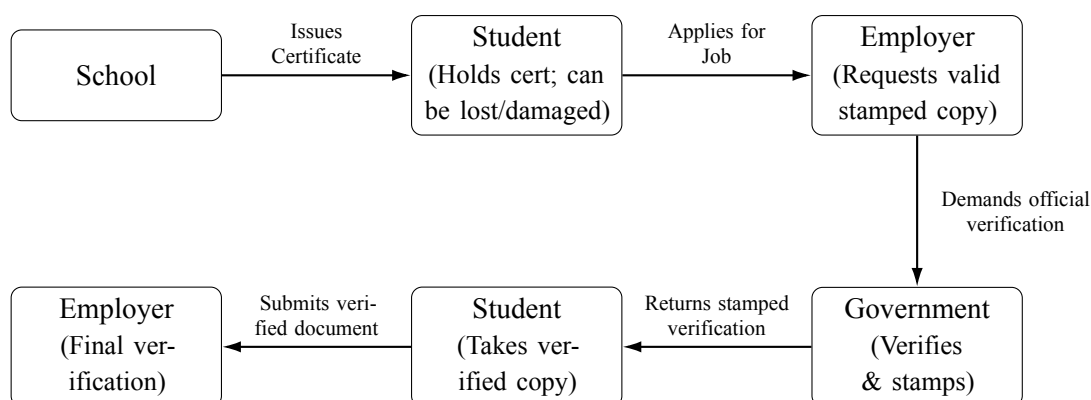


Figure 8: Traditional Certificate Verification Process Flow

- **Centralized Database Systems:** To speed things up, most modern institutions migrated records to central digital repositories. Yet this migration just swapped physical vulnerabilities for a digital Single Point of Failure (SPoF)[2]. When the central server goes down, the university's entire verification capability vanishes with it. Worse, central ledgers are only as honest as the people managing them. A corrupt database admin or an employee with hijacked high-level credentials can quietly alter grades or falsify a graduation status directly in the database, rendering internal fraud incredibly difficult to track.

2.5.2 Blockchain-based Verification Architectures

Blockchain architectures fundamentally rebuild this trust model. Instead of relying on the administrative integrity of a single institution, they shift credential authentication away from database access and ground it directly in cryptographic proof[25].

Distributed Record Storage : Rather than locking student records inside a solitary university server room, a blockchain-based setup anchors academic milestones to a peer-to-peer network[26]. The moment a university logs a certificate status on the ledger, that entry is copied across dozens or hundreds of independent nodes simultaneously. This redundancy keeps the verification pipeline up and running; even if a massive cyberattack takes the issuing university’s internal servers completely offline, third-party employers can still securely verify student records anywhere in the world[25].

Asymmetric Cryptography and Tamper-Evidence : At its core, the integrity of these records relies on public-private cryptographic key pairs. When an institution issues a diploma or logs an educational milestone, it must digitally sign that transaction using its private key[26].

- **Proof of Origin:** Because the university binds its public key to its public identity on the network, an employer’s background-check software can instantly verify exactly where the credential came from. There is no need to pick up the phone or send verification emails to a registrar’s office[25].
- **Tamper-Evidence:** The data is practically impossible to alter secretly. If a rogue administrator or external hacker attempts to modify a historical record, the tweak breaks the cryptographic hash of that specific block. The network’s consensus mechanism spots this structural mismatch instantly and drops the fraudulent change on the spot[26].

Student Ownership and Tokenization : Traditional ecosystems force the university to play the role of permanent custodian and gatekeeper for student data. Blockchain subverts this dependency, shifting data ownership over to the graduates themselves[25]. Once a school transfers a certificate token to a student’s personal wallet address, that student holds absolute, exclusive control over their academic proof via their private key[26]. Even if the university downscales, changes its entire IT infrastructure, or shuts its doors permanently, the graduate’s credential remains valid and universally verifiable[25]. The underlying framework achieves this by tokenizing degrees into unique, non-fungible assets that leverage the ERC-721 properties detailed in Section 2.4—embedding permanent institutional metadata straight into the student’s cryptographic wallet.

Architectural Dimension	Traditional Centralized Systems	Blockchain-based Systems
Data Storage Topology	Concentrated on localized, internal relational databases (SPOF)	Replicated uniformly across a distributed, global P2P node network

Verification Latency	High; requires manual processing, third-party audits, or administrative outreach	Instantaneous ($O(1)$ lookup efficiency) via programmatic cryptographic verification
Data Security & Integrity	Vulnerable to internal database tampering, unauthorized SQL overrides, and cyberattacks	Vulnerable to internal database tampering, unauthorized SQL overrides, and cyberattacks
Asset Custody	Maintained exclusively by the host university's IT infrastructure	Self-sovereign; securely held in the graduate's personal cryptographic wallet
Operational Availability	Vulnerable to server downtimes, institutional restructuring, or network blockades	Permanently accessible globally with guaranteed uptime via decentralized execution

Table 1: Traditional Centralized Systems vs Blockchain-based System

2.6 Existing Research Papers

Before discussing the proposed solution, it is important to examine how previous researchers have addressed the problem of academic credential verification using blockchain technology. Existing studies can generally be grouped into two categories: hash-based certificate verification systems and NFT-based credential management systems [27].

The below table in the next section provides a comparison of these studies, including their main approaches, contributions, and limitations. This review helps identify the research gaps that the proposed system aims to address.

2.6.1 Literature Review Matrix

Authors	Short Description	Limitations
MIT Media Lab (2016) [28]	<i>Blockcerts Framework</i> : Generates local cryptographic hashes of student certificates and batches them into a Merkle Tree structure, publishing the Merkle root hash via a standard transaction string on the public Bitcoin network.	1. Exorbitant Transaction Costs: Batching logic on the Bitcoin mainnet incurs volatile and prohibitive network gas fees. 2. Lack of Granular Control: Does not natively support independent token identities or programmatic individual revocations.
Muhammed et al. (2019) [29]	<i>EduCTX System</i> : A decentralized higher education platform utilizing a private, consortium-based Hyperledger Fabric network to log ECTS credits and diploma metadata on a shared ledger.	1. Interoperability Deficit: Being an isolated private network, external HR platforms and global applications cannot easily access or auto-verify records. 2. Scope Limitation: Limited strictly to standard higher education structures; fails to scale for non-academic professional credentials.
Zhao et al. (2022) [30]	<i>NFTCert System</i> : A private blockchain-based NFT architecture designed to mask student identities by omitting personal identifiers from on-chain metadata, storing records natively inside localized digital wallets.	1. Regulatory Barriers: The architecture mandates cryptocurrency-based gas models for transactional execution. This requirement conflicts directly with strict regulatory payment restrictions enforced across multiple international jurisdictions.
Rahman et al. (2023) [31]	<i>Verifi-Chain Protocol</i> : Integrates an Ethereum public blockchain network with the InterPlanetary File System (IPFS). Digital degree components are compiled off-chain to IPFS to extract unique CIDs, which are recorded as hashes inside public ledger nodes.	1. Query Latency Vulnerability: The prototype did not implement an active index lookup optimization, creating bottleneck delays during high-volume certificate searches. 2. Throughput Evaluation Gaps: Performance metrics concerning network data latency and transaction throughput were omitted.

Khati et al. (2023) [32]	<i>NFT Provenance Sharing Model</i> : Proposes a unified, single-platform architecture where student course components from multiple separate institutions are aggregated and minted as verifiable NFT metadata elements utilizing IPFS storage layers	1. Manual Minting Dependency: The framework relies on the student explicitly logging in to filter courses and manually execute metadata minting requests. 2. Lacks AI Automation: Contains no natural language automated verification layers or intelligent assistant interfaces for rapid third-party credential auditing.
Priyadarshini et al. (2025) [33]	<i>Hash-Mapped Authentication Architecture</i> : Implements proxy re-encryption (PRE) paired with a specialized hash function mapping interface to expedite searching algorithms across Ethereum node clusters.	1. Integration Rigidity: The software schema exhibits significant compatibility friction when interfacing with legacy university IT ecosystems and traditional data architectures. 2. Horizontal Domain Isolation: The cryptographic structure was constructed strictly for healthcare domain records (e.g., medical certificates) and cannot easily scale or adapt to handle academic grading parameters.

Table 2: Literature Review Summary of Blockchain Frameworks

2.6.2 Identified Research Gaps and Proposed Technical Solutions

An review of the existing literature highlights several limitations that remain insufficiently addressed by current blockchain-based academic credential systems. The proposed solutions aims to address these challenges through the system design presented in the list below:

1. *Balancing Cost and Interoperability*:

Many existing solutions are deployed either on public blockchain networks, which may incur transaction fees, or on private blockchain platforms that offer lower operating costs but limited interoperability with external systems. As a result, there is often a trade-off between accessibility, cost, and public verifiability.

⇒ To address this issue, the proposed system is developed and tested on the Ethereum ecosystem using the Sepolia test network for public blockchain deployment and Hardhat for local development and testing. This approach provides compatibility with widely used blockchain tools while avoiding transaction costs during development.

2. *Certificate Transferability:*

Academic certificates represent personal achievements and should remain associated with the individual who earned them. However, standard ERC-721 NFTs are transferable by design, allowing ownership to be moved between wallet addresses.

⇒ The proposed system modifies the standard ERC-721 implementation by disabling transfer operations. As a result, certificate NFTs function as Soulbound Tokens (SBTs), ensuring that ownership remains permanently linked to the graduate's wallet address.

3. *Usability of Verification Systems:*

Several existing blockchain-based verification platforms require users to interact directly with blockchain explorers or understand technical blockchain concepts. This can create challenges for university staff, employers, and other non-technical users who simply need to verify the authenticity of a certificate.

⇒ To improve usability, the proposed system includes an AI-assisted verification interface that allows users to interpret verify result to easier to understand through a simplified web application. The AI-agent support to use natural language interaction to make the verification process more accessible to non-technical users.

The proposed system addresses these limitations by combining a non-transferable NFT certificate model with a user-friendly verification platform built on blockchain and decentralized storage technologies.

3 SYSTEM ANALYSIS & DESIGN

The previous chapters discussed the fundamental concepts that form the basis of this project, including blockchain technology, smart contracts, decentralized storage, and Soulbound Tokens. Building on these concepts, this chapter presents the design and implementation of the proposed blockchain-base NFTs for academic certificates verification system.

The chapter begins by examining the limitations of traditional certificate management and verification methods. It then describes the proposed system architecture, including the integration of smart contracts, IPFS-based storage, and an AI-assisted verification interface. Finally, the operational workflow of the system is presented to illustrate how academic certificates are issued, stored, and verified in a decentralized environment.

3.1 Proposed System

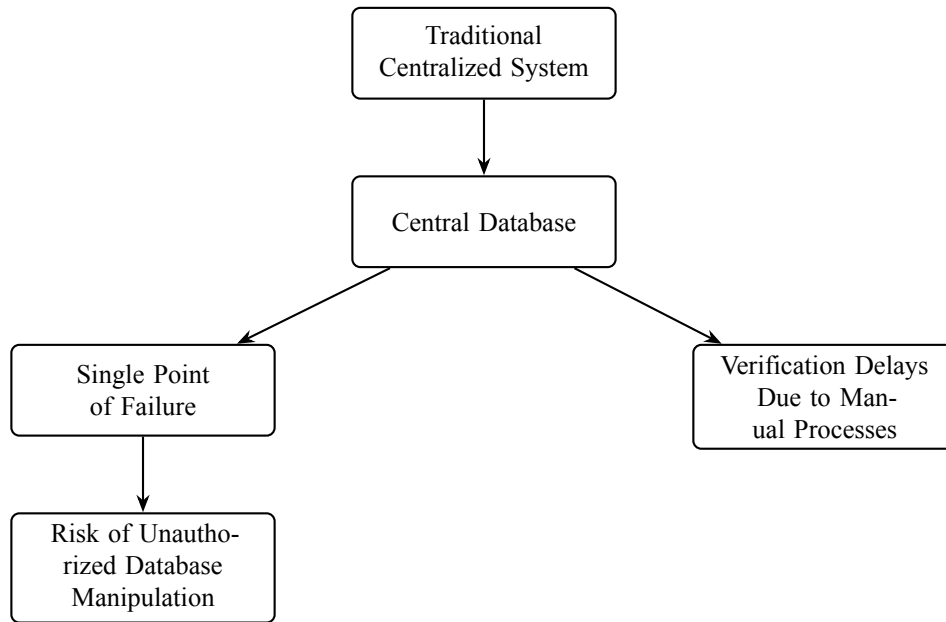


Figure 9: Limitations of Traditional Centralized Certificate Systems

The proposed solution is a blockchain-based academic credential verification system that issues academic certificates as non-transferable digital certificates tokens inspired by Soulbound Token (SBT) concepts. The system combines smart contracts, decentralized storage, and an AI-assisted verification interfaces to improve certificate authenticity, transparency, and accessibility.

Unlike traditional centralized management systems that rely on centralized databases, the proposed architecture records certificate ownership and verification data on a blockchain network while storing certificate metadata on IPFS. The overall workflow of the system is shown below:

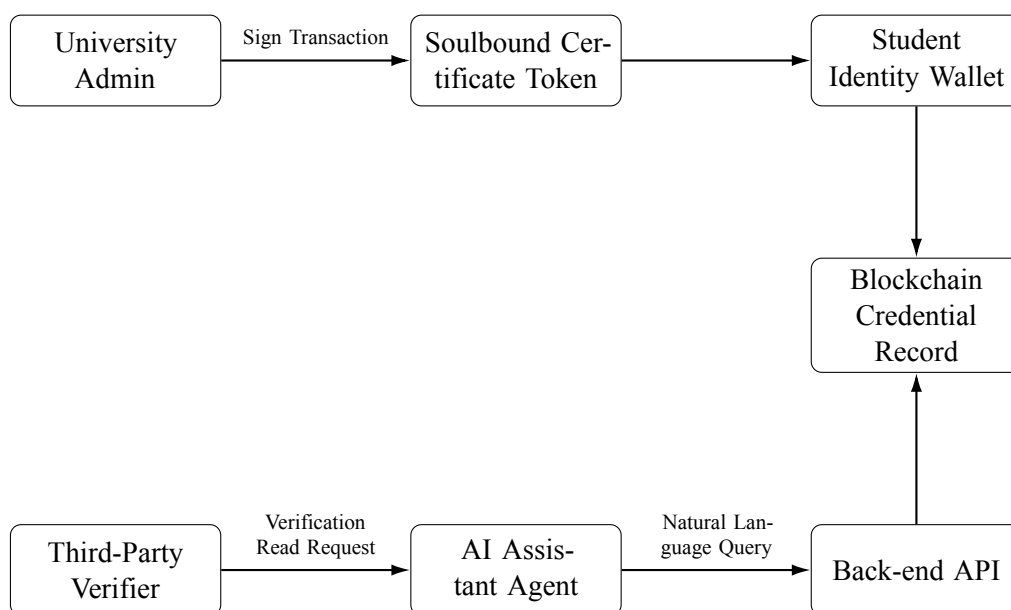


Figure 10: Operation workflow

3.1.1 Core Engineering Components

Cryptographic Certificate Issuance: When a student graduates, the university administrator generates a digital certificate and initiates the issuance process through the system. The smart contract then creates a unique certificate non-transferable certificate NFT and assign it to the student's wallet address. Each NFT contains a reference to metadata stored on IPFS, allowing the certificate information to be retrieved and verified when needed.

Non-Transferable Ownership Logic: Academic certificates should remain permanently associated with the individuals who earned them. To prevent certificate trading, identity swapping, or unauthorized ownership transfer, the smart contract disables NFT transfer operations. Once a certificate is issued to student's wallet, ownership cannot be transferred to another address. This approach follows the principles of Soulbound Tokens (SBTs) and helps prevent certificate trading, identity misuse, and unauthorized ownership changes.

Decentralized Metadata Storage: Storing large files directly on-chain is expensive and inefficient. Therefore the system stores certificate documents and metadata on IPFS while keeping only the corresponding Content Identifier (CID) on the blockchain. This approach reduces storages costs while preserving document integrity and traceability.

3.1.2 Addressing Existing Limitations

Based on the literature review, the proposed system addresses several limitations found in existing academic credential verification solutions.

- **Low-Cost Development and Testing:** For development, testing, and validation, the system is develop using the Hardhat development environment and configured for deployment on the Sepolia Ethereum Test Network, allowing testing and validation without incurring real transaction costs.
- **Non-Transferable Credentials:** Unlike standard NFT implementations, the proposed certificate NFTs cannot be transferred between users. This ensures that academic credentials remain permanently linked to their original owners.
- **Transparent Verification:** Certificate ownership and validity can be verified directly from blockchain records. This reduces dependence on centralized institutional databases and improves long-term accessibility.
- **AI-Assisted Accessibility:** The system includes an AI-based assistant that helps users interpret verification results and navigate the verification process through natural language interactions. This makes the platform more accessible to users without blockchain knowledge.

3.1.3 Certificate Verification Workflow

The proposed system verifies academic credentials using combining cryptographic hashing, blockchain records, and decentralized metadata storage.

During certificate issuance, the university generates a certificate and calculates its SHA-256 hash. This hash acts as a unique digital fingerprint of the document. The hash is then linked to

a certificate NFT and stored in the smart contract.

During verification, an external verifier uploads the certificate document. The system calculates a new SHA-256 hash from the uploaded file and compares it with the hash stored on the blockchain.

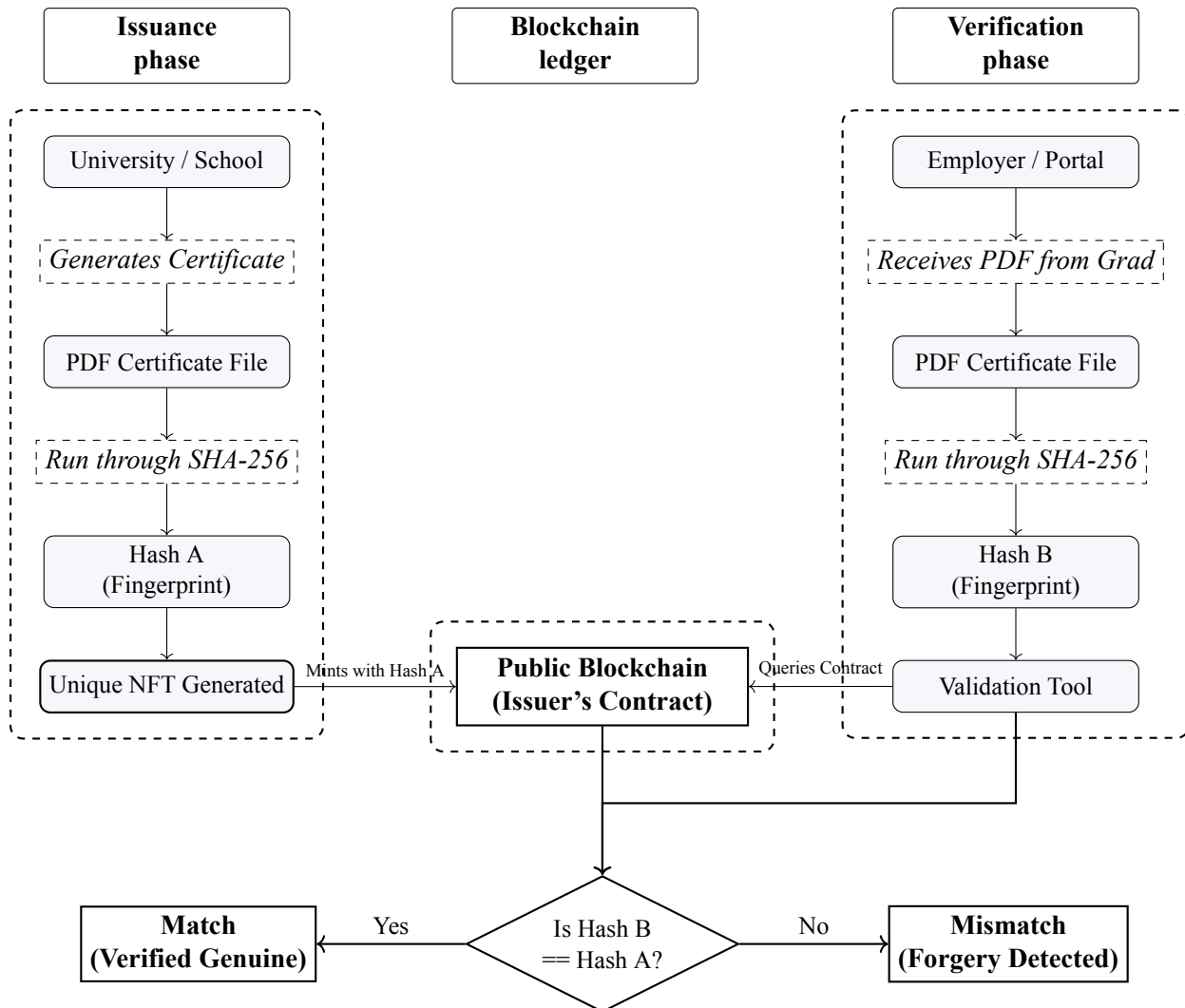


Figure 11: Decentralized Academic Certificate Verification Workflow

Let breakdown the above diagram:

- **Issuance Phase:** At this stage, the university creates the digital certificate asset. Before storage and token issuance, the file is processed through the SHA-256 hashing algorithm, creating a unique, immutable 64-character hexadecimal string (in the diagram, Hash A). The smart contract creates an NFT containing this specific hash string in its metadata structure.
- **Verification phase:** When applicants submit their applications, they send a digital PDF file to the employer. The verification process runs the same SHA-256 algorithm as on the file the applicant uploaded to calculate a unique hash code (Hash code B in the diagram).
- **Certificate Integrity Verification:** The blockchain network query platform reads hash code A stored in the token registry. If Hash A == Hash B, the document structure is com-

pletely intact and verified as correct. If even a single letter, dot, or pixel differs, the hash code will result in a final result of No Match (Forgery Detection).

3.2 Architecture Diagram

The proposed system follows a multi-layer architecture that separates the user interface, backend services, blockchain network, and decentralized storage components. This separation improves maintainability, scalability, and security by allowing each layer to operate independently while still working together as a complete system.

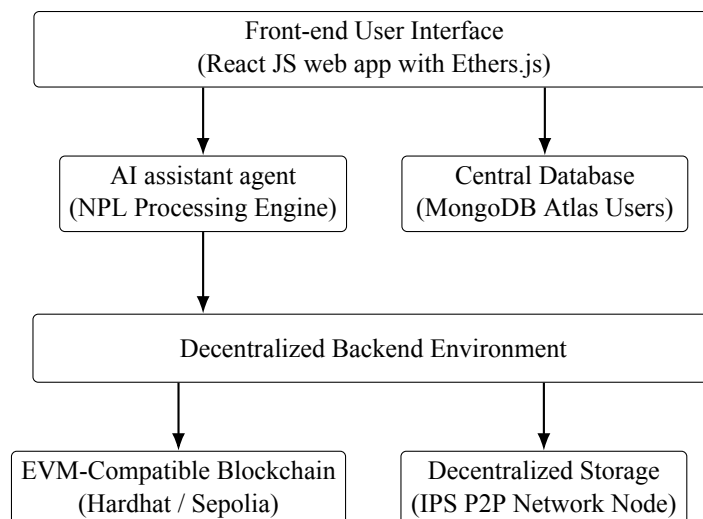


Figure 12: Architecture Diagram

Front-end User Interface	On the frontend, we will use React JS, which acts as the client for administration, wallet authentication, and student profile management. It communicates with web3 provider wrappers (Ethers.js) to execute transaction payloads.
AI Assistant Agent	The AI assistant serves as an intelligent support component that helps users understand certificate information, verification results, and system functionalities. It retrieves relevant data from the backend services and presents responses in a user-friendly manner, reducing the need for users to directly interact with blockchain explorers or technical verification tools.
Central Database	To manage off-chain application state, user authentication records, session configurations, and temporary system operation configurations, we will use MongoDB to store the data in the cloud. It does not store primary authentication data

Blockchain Layer	The cryptographic core is built using Solidity. It serves as the single source of truth, maintaining token registries, validation mappings, and enforcement parameters across distributed validation nodes.
Decentralized Storage (IPFS)	A peer-to-peer, content-addressed storage repository hosting the static JSON schemas and cryptographic certificate files

Table 3: Architecture diagram explanation

3.3 Use Case Diagram

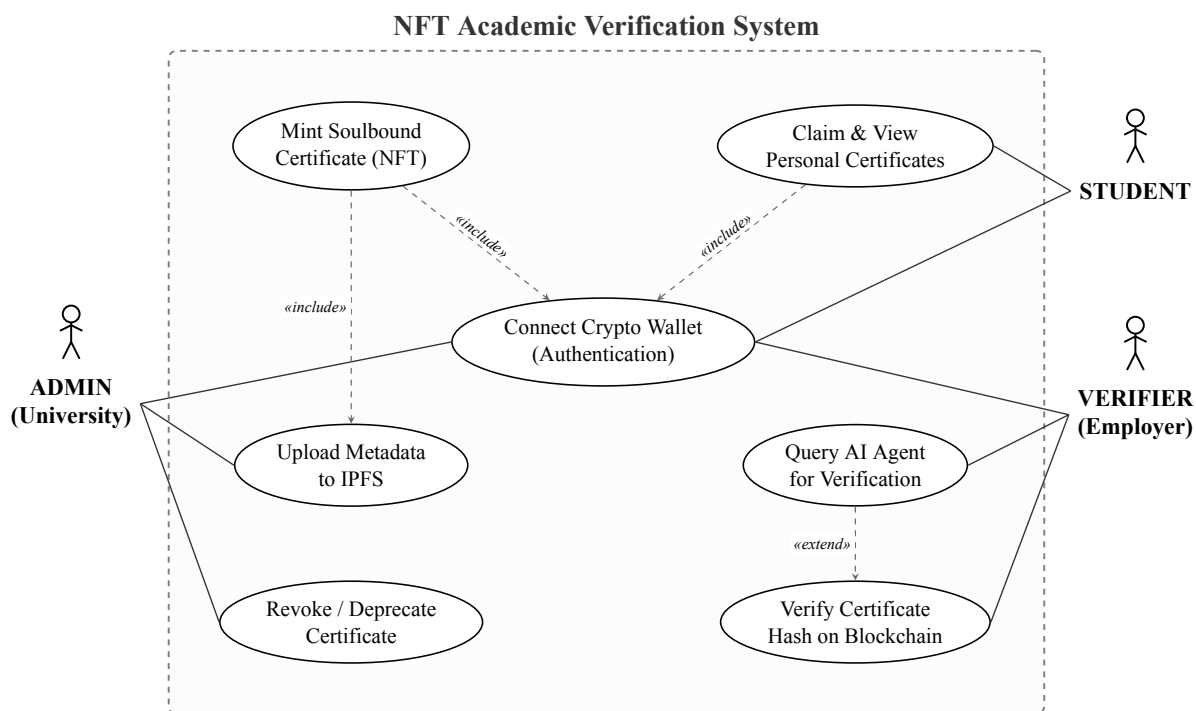


Figure 13: Use Case Diagram

Actor	Description
Administrator (University Registrar)	• Authentication: Connects the university wallet to access the administrative dashboard. • Issue certificate: Creates digital certificates, uploads certificate data to IPFS, and initiates the NFT minting process through the smart contract • Revoke certificate: Marks a certificate as revoked when necessary while preserving the original record on the blockchain.
Student	• Authentication: Connects a personal wallet to access the system. • View certificate: Accesses personal academic credentials, certificate details, and blockchain verification information.
Verifier	• Verification request: Uses the web interface or AI assistant to verify a certificate. • Certificate validation: Checks certificate authenticity, ownership information, and verification status using blockchain records and IPFS metadata.

Table 4: Description of system actors and responsibilities

3.4 Sequence Diagram

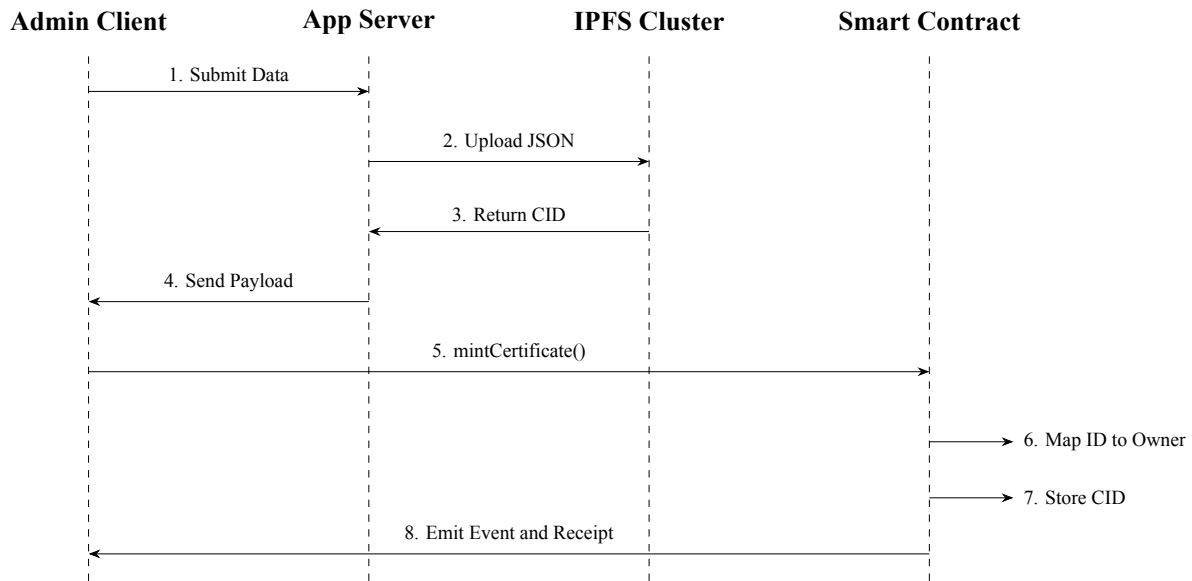


Figure 14: Sequence Diagram

The above figure illustrates the sequence of interactions between the administrator, application server, IPFS, and blockchain network during the certificate issuance process.

1. The admin submits the student's academic information through the system interface and the frontend call to the application server backend.
2. The backend server formats the certificate data into a structured JSON schema configuration and uploads it to the IPFS network cluster.
3. The metadata is uploaded to IPFS, which generate a unique, tamper-evident Content Identifier hash (CID) and pass it back to the server.
4. The server relays the prepared transaction parameters along with the CID back to the Admin client interface.
5. The Admin signs and transmits an EVM transaction payload calling the `mintCertificate(studentAddress, tokenURI)` function on the deployed smart contract.
6. The smart contract validates the Admin's address and updates the internal mapping index array, binding the new unique token ID directly to the target student's public wallet address.
7. The smart contract records the immutable IPFS CID string into persistent blockchain storage.
8. The network executes state confirmation, writes the block to the distributed ledger, emits a Minted logging event, and returns a transaction receipt to the frontend client.

3.5 Activity Diagram

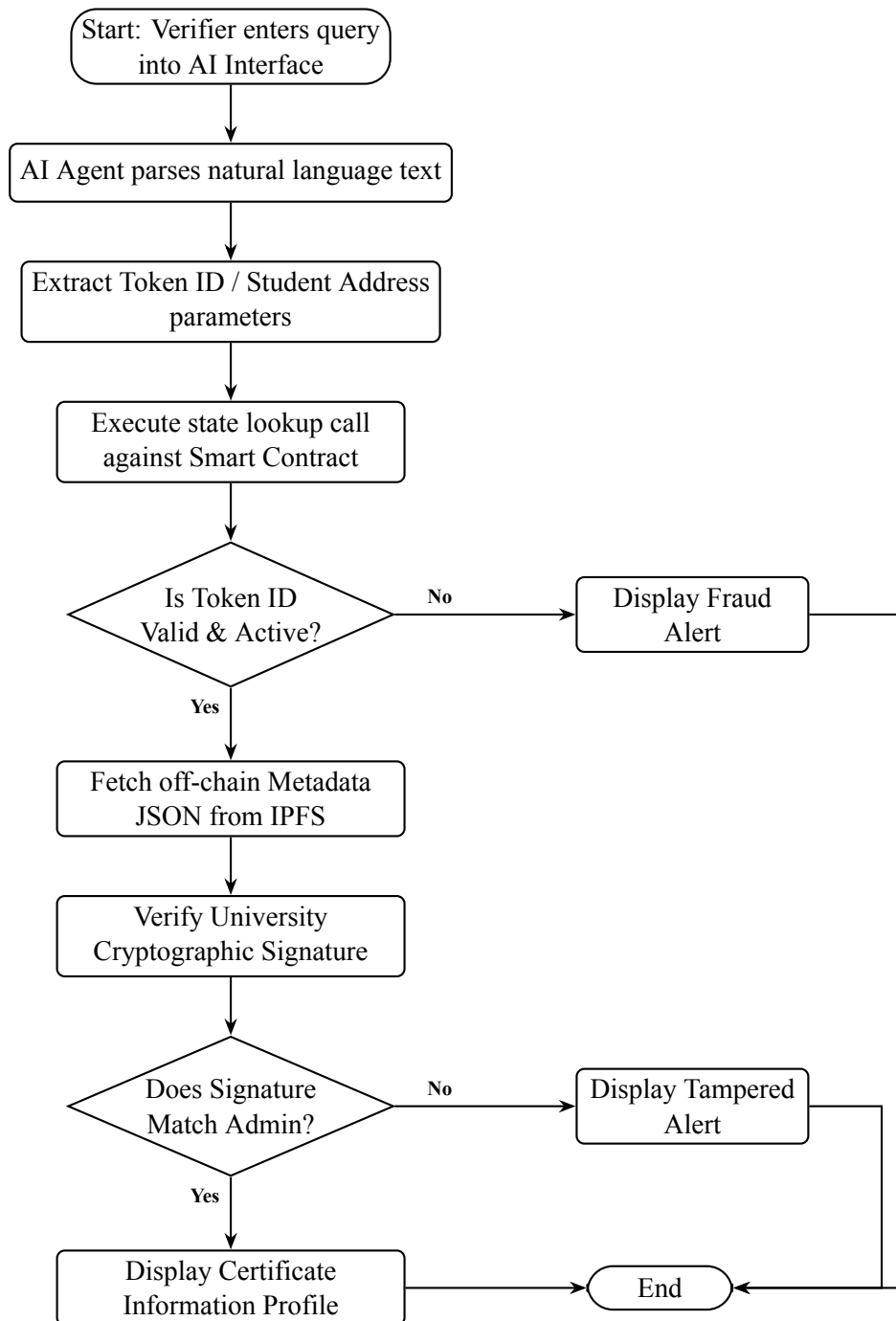


Figure 15: Activity Diagram

3.6 Database Diagram

The proposed system uses both blockchain storage and a traditional database. The blockchain is responsible for storing certificate ownership and verification information, while MongoDB manages application-level data such as student records, certificate records. This hybrid approach improves efficiency by keeping frequently updated information in the database while preserving certificate integrity on the blockchain.

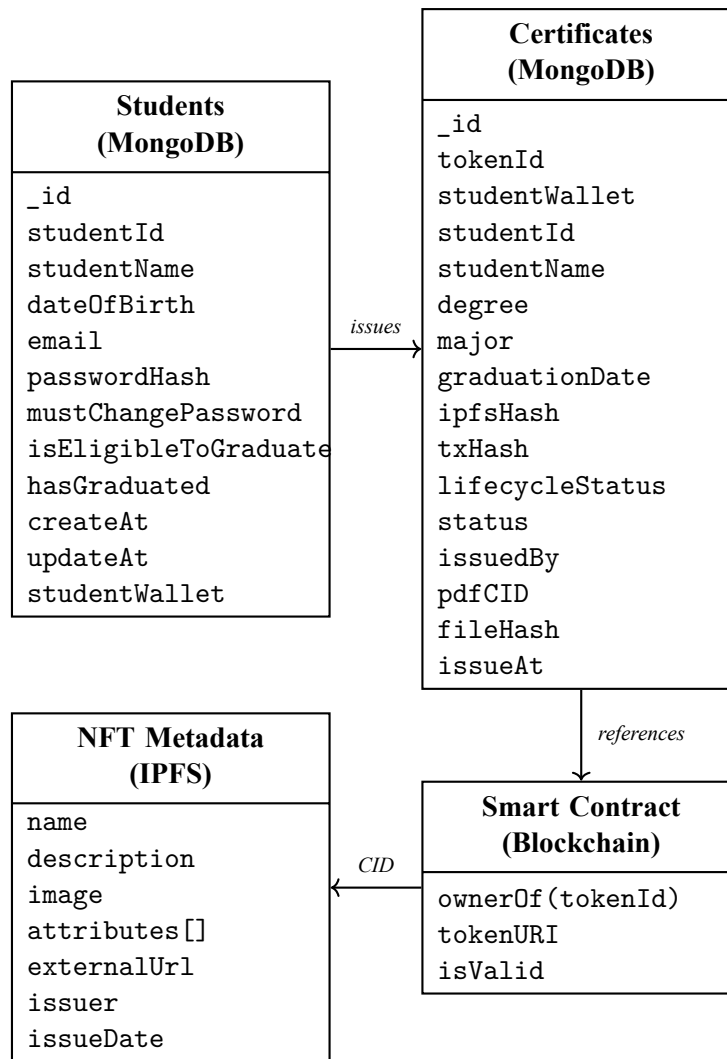


Figure 16: Database Diagram

Listing 1: Users Collection

```

{
  "_id": "ObjectId('65f3a1b2e4b0c2a1d4e5f6a1')",
  "walletAddress": "0x71C7656EC7ab88b098defB751B7401B5f6d1476B",
  "role": "Admin",
  "fullName": "Office of the Registrar",
  "email": "registrar@university.edu",
  "status": "Active",
  "createdAt": "2026-01-15T08:30:00.000Z"
}
  
```

Listing 2: Certificates Collection

```

{
  "_id": "ObjectId('65f3a1b2e4b0c2a1d4e5f6a2')",
  "tokenId": 1042,
  "studentWallet": "0xABC123...",
  "studentName": "Nguyen Van A",
  "degreeType": "Bachelor of Science",
  "major": "Computer Science",
  "graduationDate": "2026-05-28",
}
  
```

```

    "ipfsHash": "QmZtm26T...H6Kx7z",
    "txHash": "0x9f8e7d...6b5a4c",
    "status": "Active",
    "issuedBy": "0xAdminWallet",
    "issuedAt": "2026-05-28T10:00:00Z",
    "revokedAt": null,
    "revocationReason": null
  },
  {
    "_id": "ObjectId('65f3a1b2e4b0c2a1d4e5f6a2')",
    "tokenId": 1042,
    "studentWallet": "0xABC123...",
    "studentName": "Nguyen Van A",
    "degreeType": "Bachelor of Science",
    "major": "Computer Science",
    "graduationDate": "2026-05-28",
    "ipfsHash": "QmZtm26T...H6Kx7z",
    "txHash": "0x9f8e7d...6b5a4c",
    "status": "Revoked",
    "issuedBy": "0xAdminWallet",
    "issuedAt": "2026-05-28T10:00:00Z",
    "revokedAt": "2027-01-10T08:00:00Z",
    "revocationReason": "Administrative correction"
  }
}

```

Listing 3: NFT Metadata Collection

```

{
  "name": "Academic Degree Certificate #1042",
  "description": "Official digital verification document issued by State Technical University.",
  "image": "ipfs://QmYwAP...xZ2tP/certificate.pdf",
  "attributes": [
    { "trait_type": "Student Name", "value": "Nguyen Van A" },
    { "trait_type": "Degree ID", "value": "2026-CS-1042" },
    { "trait_type": "Major", "value": "Computer Science" },
    { "trait_type": "GPA", "value": "3.85" },
    { "trait_type": "Honors", "value": "Magna Cum Laude" }
  ],
  "externalUrl": "https://university.edu/verify/1042",
  "issuer": "State Technical University",
  "issueDate": "2026-05-28"
}

```

3.7 Smart Contract Design

The proposed academic certificate verification system utilizes a Solidity smart contract deployed on an EVM-compatible blockchain network. The smart contract serves as the decentralized registry responsible for issuing, validating, and managing academic credentials represented as Soulbound Tokens (SBTs).

Instead of storing complete certificate information on-chain, the contract stores a reference to certificate metadata hosted on IPFS. This approach reduces blockchain storage requirements while preserving transparency, immutability, and verifiability. The contract maintains certificate

ownership information, validity status, and administrative permissions required for credential management.

3.7.1 Core Functions

The smart contract provides three primary functions that manage the lifecycle of an academic certificate.

1. `mintCertificate(address student, string memory tokenURI)`

This function is used by the university administrator to issue a new academic certificate. The function generates a unique Token ID, associates it with the student's wallet address, stores the IPFS metadata reference, and marks the certificate as valid.

2. `verifyCertificate(uint256 tokenId)`

This public read-only function allows external users and verification systems to validate a certificate. Given a Token ID, the function returns the certificate owner, metadata URI, and validity status.

3. `revokeCertificate(uint256 tokenId)`

This administrative function invalidates an existing certificate by changing its status from active to inactive. Revocation may be required in cases such as administrative corrections or invalid credential issuance.

3.7.2 Soulbound Enforcement

Unlike traditional ERC-721 NFTs, academic credentials should remain permanently associated with their original owners. To enforce this requirement, the proposed system adopts the Soulbound Token (SBT) model. The contract overrides the standard ERC-721 transfer functions (`transferFrom()` and `safeTransferFrom()`) and forces them to revert whenever a transfer attempt is made. As a result, certificates cannot be bought, sold, exchanged, or transferred between wallet addresses. This mechanism ensures that each certificate remains permanently linked to the graduate who originally received it, preserving authenticity and preventing credential misuse.

4 IMPLEMENTATION

4.1 Development Environment

The implementation of the decentralized academic certificate platform follows a decoupled, full-stack web3 architecture. To ensure robust performance, type safety, and real-time public ledger synchronization, the development lifecycle was executed using an ecosystem divided into five core layers:

- Frontend UI
- Backend Middleware
- Blockchain - Smart Contract Environment
- Centralized Cloud Database

- Decentralized Content Storage

Front-end	• React (^19.2.6) & Vite (^8.0.12): Used to build a modular, single-page client app. Vite serves as the build engine to achieve rapid Hot Module Replacement (HMR) and optimized bundle rendering. • TypeScript (~6.0.2): Enforced static typing constraints across all user views, catching data-type mismatches before compiling. • Axios (^1.17.0): Used to send asynchronous REST API queries to the backend server. • Ant Design (^6.4.3): Deployed as the enterprise-grade UI component library.
Back-end & Database	• Node.js (v24.x) & Express.js (^5.2.x): Used to listen for inbound front-end traffic and process operational request loops. • Typescript (^6.0.x): Maintains architectural type-safety uniformity across controller modules and middleware routes. • MongoDB Atlas & Mongoose ODM (^9.6.x): Cloud NoSQL data tier paired with an Object Data Modeling wrapper to manage off-chain application states, student accounts, and certificate information. • Ethers.js (^6.16.0): Embedded into the server layer to turn the backend into a Web3 client, opening programmatic channels to interact with local contract functions. • nodemailer(^8.0.10): Support for admin send the email to student function. • pdfkit(^0.18.0): Support for create PDF certificate file. • qrcode(^1.5.4): Support for generating QR code for quick scanning on the certificate. • groq-sdk(^1.2.1): Groq AI Agent for intepret verification certificate.

Local Blockchain Execution	• Solidity (0.8.20): Language to write our project contract. • Hardhat Framework (^2.22.x): Deployed as the comprehensive development suite used to compile Solidity contracts, run unit tests, and trace transaction gas limits. • Hardhat Network Node (http://127.0.0.1:8545): Acts as the localized private JSON-RPC network gateway. The smart contract is hosted entirely in local memory, bypassing public testnet dependencies for rapid testing. • @openzeppelin/contracts (^4.9.x): Imported into the contract development loop to provide secure, audited, industry-standard templates for core token properties.
Decentralized Storage	• IPFS via Pinata SDK: Used to securely host structural metadata JSON schemas and visual certificate templates off-chain via unique CID hashes.

Table 5: Development Environment

4.2 Smart Contract Implementation

The smart contract is responsible for issuing and storing immutable academic certificate records on the blockchain.

Listing 4: Mint Certificate

```
function mintCertificate(
    address student,
    string memory studentId,
    string memory studentName,
    string memory degree,
    string memory major,
    string memory ipfsHash,
    string memory fileHash
) public onlyOwner {
    require(hashToTokenId[fileHash] == 0, "A certificate with this file hash
    already exists");

    uint256 tokenId = ++nextTokenId;

    _safeMint(student, tokenId);

    certificates[tokenId] = Certificate(
        studentId,
        studentName,
        degree,
        major,
```

```

        ipfsHash,
        block.timestamp,
        false,
        0,
        "",
        fileHash
    );

    hashToTokenId[fileHash] = tokenId;

    emit CertificateMinted(
        tokenId,
        student,
        studentId,
        studentName,
        degree,
        major,
        ipfsHash,
        block.timestamp,
        fileHash
    );
}

```

Listing 5: Get Certificate

```

function getCertificateByHash(
    string memory fileHash
) public view returns (
    uint256 tokenId,
    string memory studentId,
    string memory studentName,
    string memory degree,
    string memory major,
    string memory ipfsHash,
    uint256 issueDate,
    bool isRevoked,
    uint256 revokedAt,
    string memory revocationReason
) {
    uint256 id = hashToTokenId[fileHash];
    require(id > 0, "No certificate matches this file hash");

    Certificate memory cert = certificates[id];
    return (
        id,
        cert.studentId,
        cert.studentName,
        cert.degree,
        cert.major,
        cert.ipfsHash,
        cert.issueDate,
        cert.isRevoked,
        cert.revokedAt,
        cert.revocationReason
    );
}

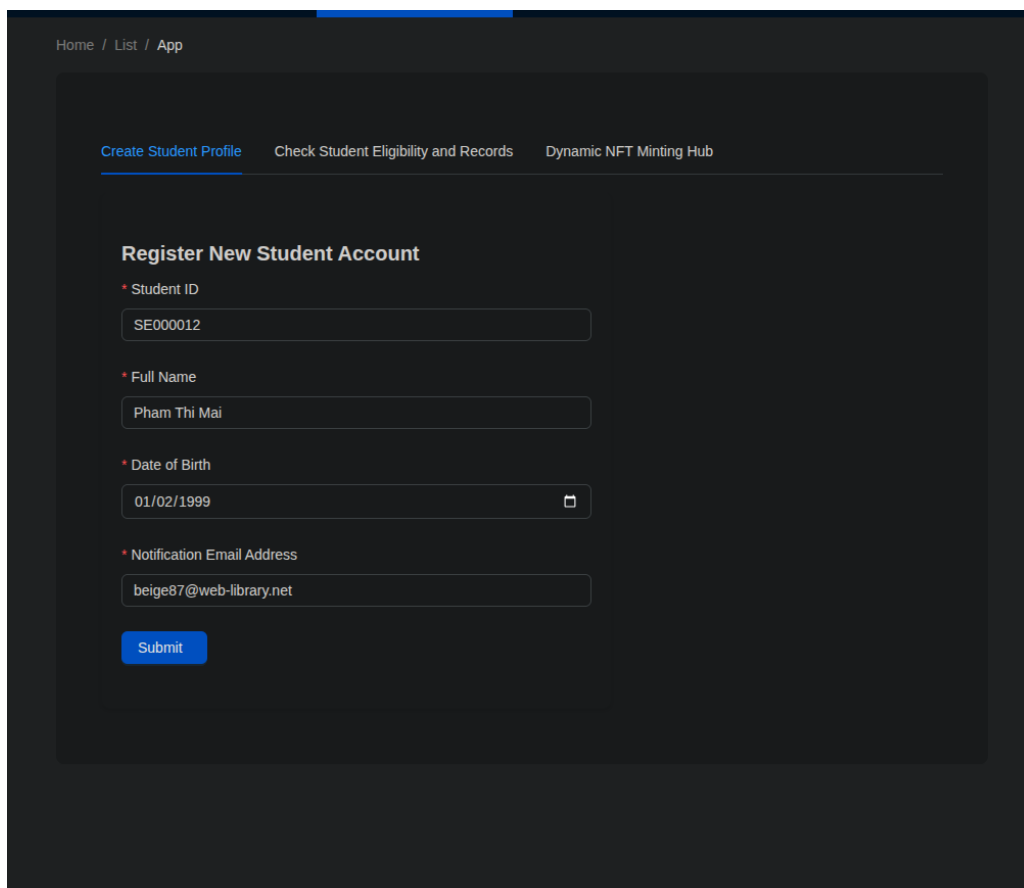
```

4.3 Front-end Implementation

The frontend interface layer is developed using React, Vite, TypeScript, and Ant Design. State encapsulation is decoupled across specific user portals to keep operational spaces separate:

4.3.1 Admin Console - Create Student Account

This screen is used for university admins to create a new student account. It will send an onboard email to the student's email address with the student ID and temporary password for login into the portal.



The screenshot displays a web application interface for creating a new student account. At the top, there is a breadcrumb trail: "Home / List / App". Below this, a navigation bar contains three links: "Create Student Profile" (which is highlighted with a blue underline), "Check Student Eligibility and Records", and "Dynamic NFT Minting Hub". The main content area is titled "Register New Student Account". It contains four form fields, each with a red asterisk indicating a required field: "Student ID" (with the value "SE000012"), "Full Name" (with the value "Pham Thi Mai"), "Date of Birth" (with the value "01/02/1999" and a calendar icon), and "Notification Email Address" (with the value "beige87@web-library.net"). A blue "Submit" button is located at the bottom of the form.

Figure 17: Admin Console - Add student account

4.3.2 Student onboard email

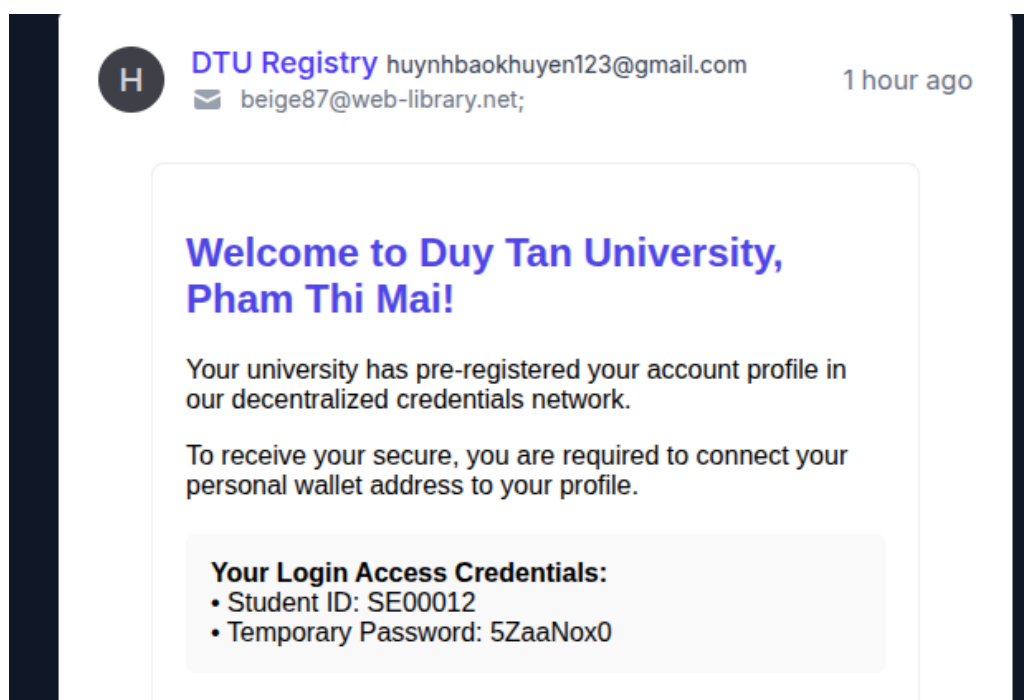


Figure 18: Student onboard email

4.3.3 Student Login

This screen used for student login into the portal

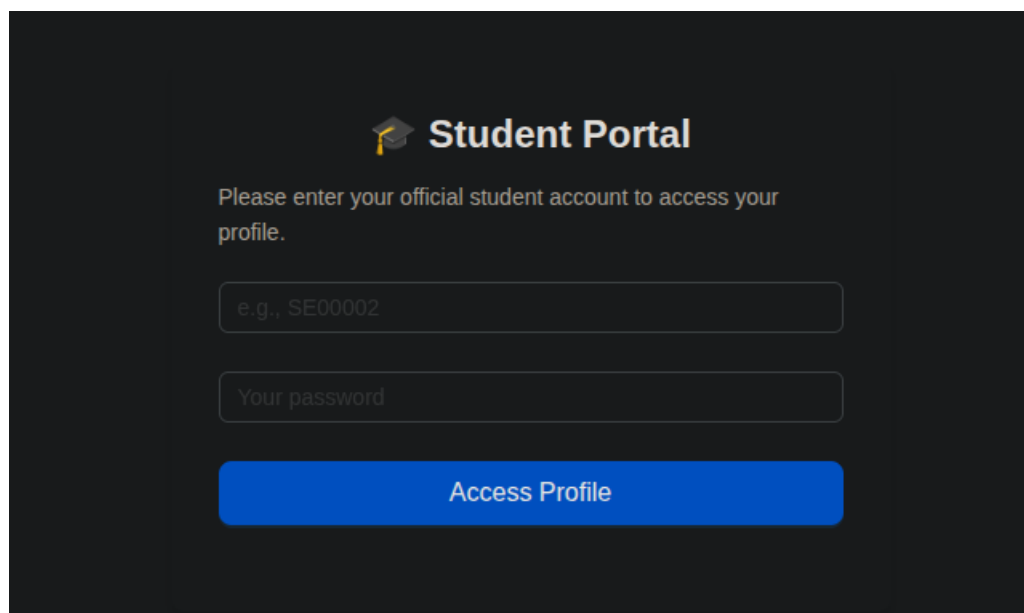
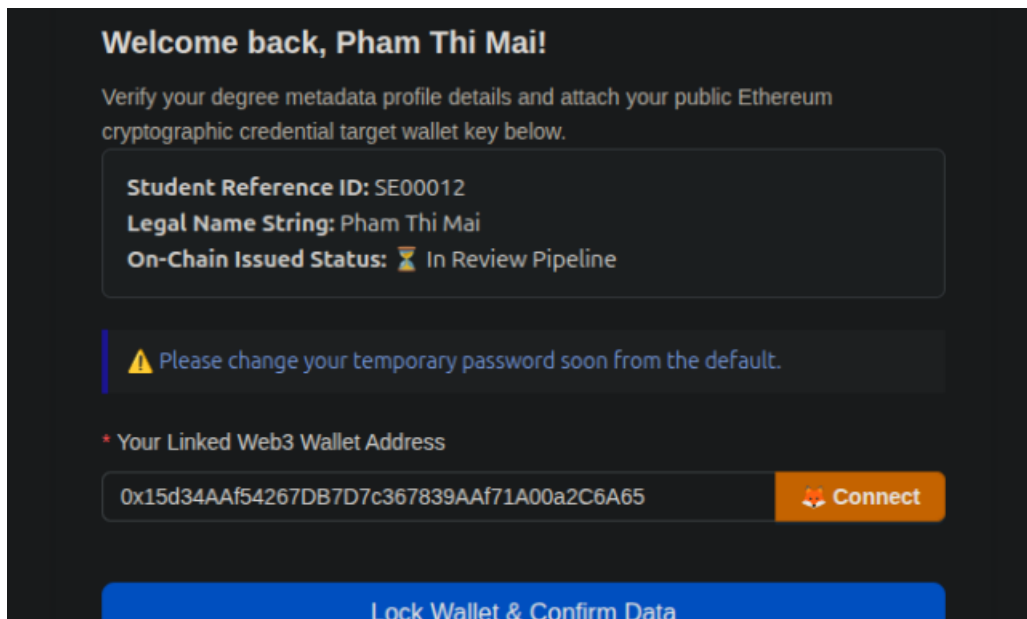


Figure 19: Student Login

4.3.4 Student submits personal wallet address

The first time a student logs into the system, they need to submit their wallet address first to save into the system. They can provide a wallet address or connect with the Matemask wallet.

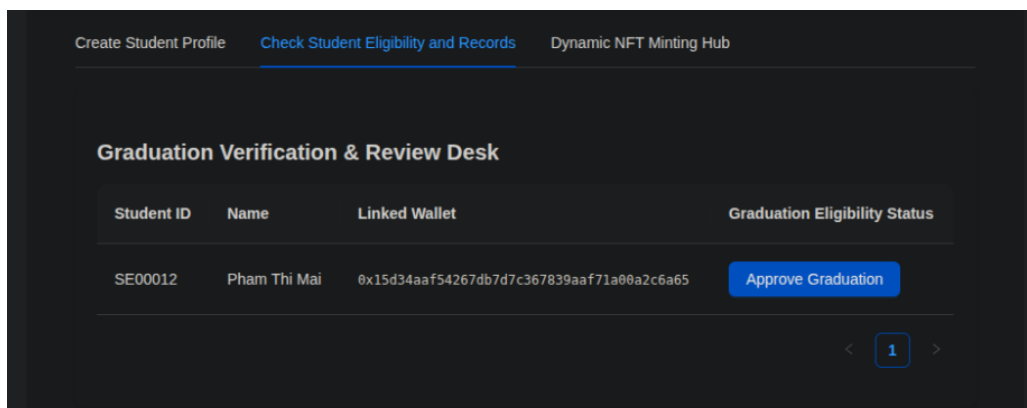


The screenshot shows a dark-themed user interface for a student named Pham Thi Mai. At the top, it says 'Welcome back, Pham Thi Mai!'. Below this, a message asks the user to 'Verify your degree metadata profile details and attach your public Ethereum cryptographic credential target wallet key below.' A box displays the user's details: 'Student Reference ID: SE00012', 'Legal Name String: Pham Thi Mai', and 'On-Chain Issued Status: ⌚ In Review Pipeline'. A yellow warning icon and text prompt the user to 'Please change your temporary password soon from the default.' Below this, a section titled '* Your Linked Web3 Wallet Address' shows a text input field containing the address '0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65' and an orange 'Connect' button with a wallet icon. At the bottom, a blue button is labeled 'Lock Wallet & Confirm Data'.

Figure 20: Student Submit Wallet

4.3.5 Admin Console - Check for graduation

In this screen, the admin can check and see if a student has enough conditions to graduate, and they can create approval for graduation.



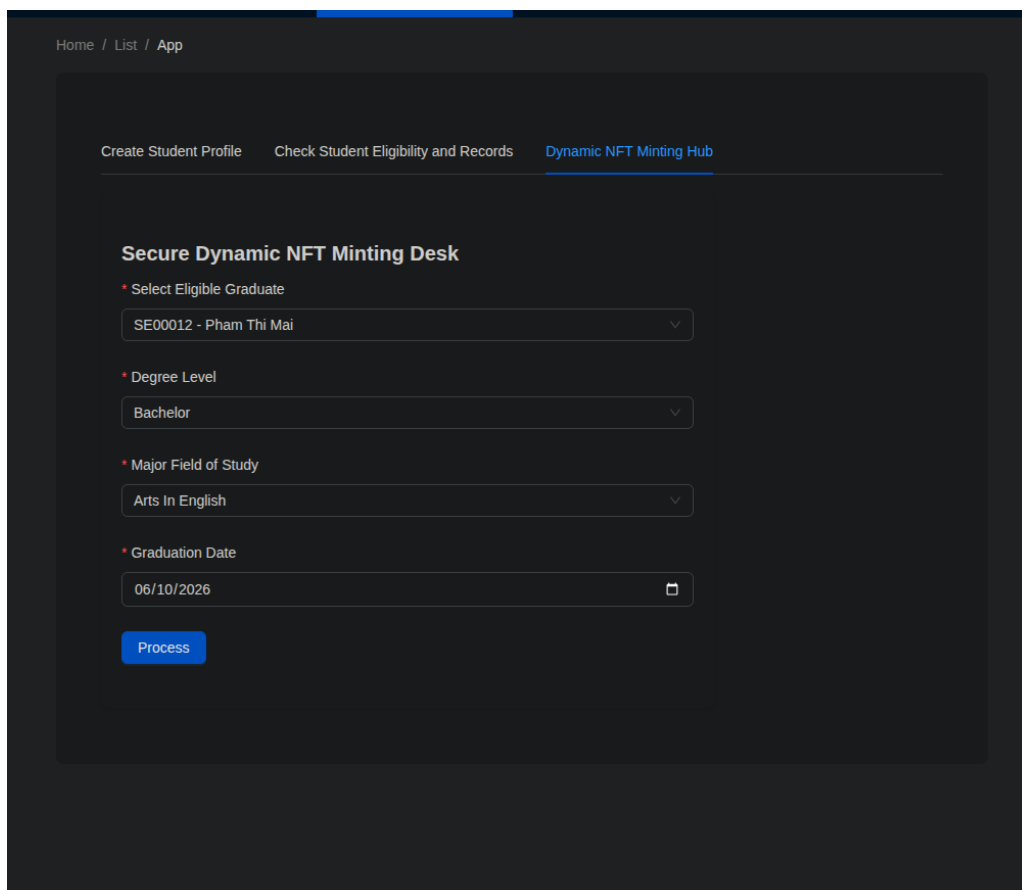
The screenshot shows the 'Admin Console - Graduation Verification & Review Desk'. At the top, there are three tabs: 'Create Student Profile', 'Check Student Eligibility and Records' (which is active), and 'Dynamic NFT Minting Hub'. Below the tabs, the title 'Graduation Verification & Review Desk' is displayed. A table lists student information with columns for 'Student ID', 'Name', 'Linked Wallet', and 'Graduation Eligibility Status'. The table contains one entry for Pham Thi Mai with ID SE00012 and a linked wallet address. An 'Approve Graduation' button is next to the entry. At the bottom right, there are navigation arrows and a page number '1'.

Student ID	Name	Linked Wallet	Graduation Eligibility Status
SE00012	Pham Thi Mai	0x15d34aaf54267db7d7c367839aaf71a00a2c6a65	<button>Approve Graduation</button>

Figure 21: Admin Console - Graduate Approve

4.3.6 Admin Console - Mint graduate certificate

After a student is approved for enough conditions to graduate, the admin can mint a certificate. It will connect to the backend api, the backend will process to connect with the IPFS and blockchain side to return the student's PDF certificate file with the hash data.



Home / List / App

Create Student Profile Check Student Eligibility and Records Dynamic NFT Minting Hub

Secure Dynamic NFT Minting Desk

* Select Eligible Graduate
SE00012 - Pham Thi Mai

* Degree Level
Bachelor

* Major Field of Study
Arts In English

* Graduation Date
06/10/2026

Process

Figure 22: Admin Console - Mint Certificate

After minting success, the admin can download the PDF certificate file. And an email will auto-send to the student's email with a PDF certificate attached.

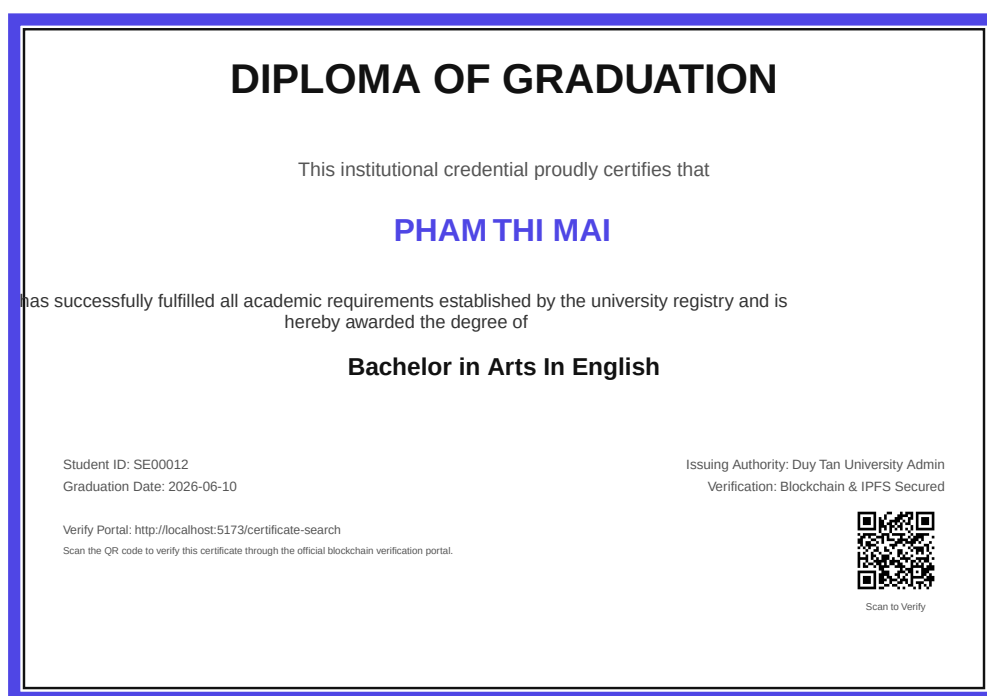


Figure 23: Admin Console - Mint Certificate - Download Button

4.3.7 Certificate Search

This public interface is accessible to all users without requiring authentication. It allows individuals to upload PDF certificates to verify their cryptographic validity against the blockchain registry. Upon processing, the system displays the verification results instantly. Furthermore, the page integrates an AI Agent designed to interpret and explain these cryptographic outcomes in a user-friendly manner

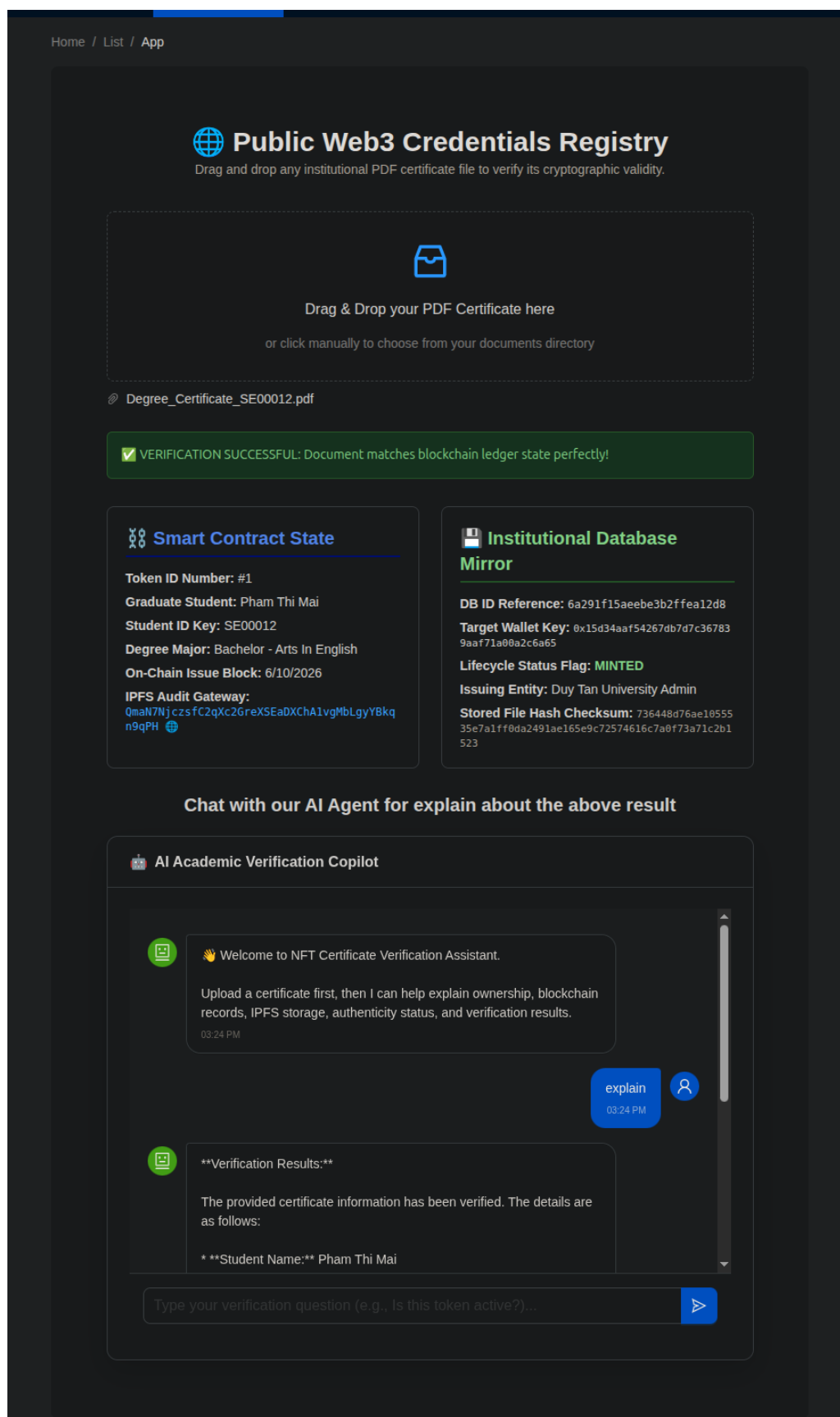


Figure 24: Verify Certificate

4.4 Back-end API

The Web2 off-chain engine is driven by a structured Node.js/Express REST API cluster. It serves as a stateless coordinator layer that translates incoming client requests into secure database and blockchain state updates.

/api/certificates/revoke/:id	Used to revoke a certificate
/api/certificates/all	Get all certificates data
/api/certificates/download/:id	Download the certificate file with pdf-CID
/api/certificates/verify-by-hash/:hash	This API used to verify the certificate PDF file is valid or not
/api/certificates/issue	Mint certificate, upload it to IPFS, save a copy of the certificate in MongoDB
/api/students/admin/register	API used for admin to create a new student account
/api/students/student/link-wallet	Save the student's wallet address in the centralized database
/api/students/admin/approve-eligibility	This API for admin approval for a student is enough condition to be graduated, it updates <code>student.isEligibleToGraduate</code> to true
/api/students/eligible-graduates	Return a list of all students with their eligible graduate status
/api/students/profile/:studentId	This API is used to return the student profile information using the Student ID
/api/students/all-profiles	This API is used to get the list of all students
/api/students/login	Login API for student
/api/students/change-password	API for changing the student password
/api/ai/verify	API for using AI-agent

Table 6: Backend APIs

4.5 IPFS Integration

To eliminate dependence on centralized file storage and improve certificate integrity, the proposed system utilizes the InterPlanetary File System (IPFS) through the Pinata gateway service. Both the generated certificate PDF and its associated metadata are stored in decentralized content-addressed storage.

When a certificate is issued, the backend first generates a PDF document containing the student's academic information.

Listing 6: Generate PDF file

```
const certificatePayload = {
  studentWallet: student.studentWallet,
  studentId: student.studentId,
  studentName: student.studentName,
  degree,
  major,
  graduationDate,
  issuedBy,
};

const pdfBuffer = await generateCertificatePDFBuffer(certificatePayload);
```

A SHA-256 cryptographic hash is then computed from the PDF content to create a unique integrity fingerprint (fileHash). The PDF file is uploaded to IPFS and returns a Content Identifier (pdfCID).

Listing 7: Create file hash and upload PDF to IPFS

```
const fileHash = crypto
  .createHash("sha256")
  .update(pdfBuffer)
  .digest("hex");

const pdfCID = await uploadPDFToIPFS(
  pdfBuffer,
  `Certificate_${student.studentId}.pdf`,
);
```

After obtaining the PDF CID, the system constructs a metadata JSON object containing descriptive information, certificate attributes, the PDF reference, and the integrity hash:

Listing 8: Metadata JSON

```
{
  "name": "Academic Certificate - Huynh Thi Luu",
  "description": "Official decentralized educational degree issued by University Administrator.",
  "pdfCID": "QmXXXXXX",
  "fileHash": "18d367c6xxxx", "attributes": [
    {
      "trait_type": "Student ID",
      "value": "SE12345"
    },
    {
      "trait_type": "Degree",
      "value": "Bachelor"
    }
  ],
}
```

```

    {
      "trait_type": "Major",
      "value": "Computer Science"
    },
    {
      "trait_type": "Graduation Date",
      "value": "2026-03-03"
    }
  ]
}

```

The metadata object is subsequently uploaded to IPFS, producing a second Content Identifier (metadataCID). During the NFT minting process, the smart contract permanently records the metadata CID and file hash on the blockchain.

Listing 9: Create file hash and upload PDF to IPFS

```

const tx = await certificateContract.mintCertificate(
  student.studentWallet,
  student.studentId,
  student.studentName,
  degree,
  major,
  metadataCID,
  fileHash,
);

const receipt = await tx.wait();

```

Because IPFS employs content-addressed storage, any modification to either the certificate PDF or metadata file results in a completely different CID value. This mechanism provides tamper resistance and enables independent verification of certificate authenticity. The resulting architecture follows a hierarchical decentralized storage model:

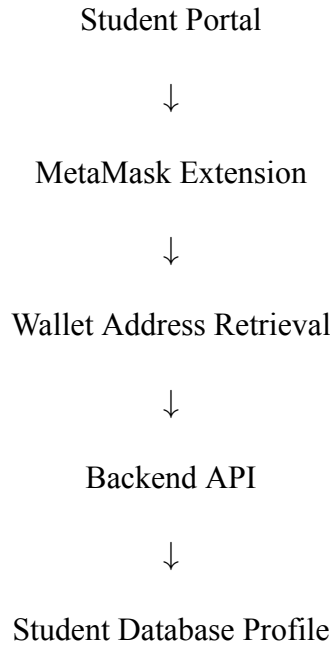
NFT → Metadata CID → Metadata JSON → PDF CID → Certificate PDF

This design separates ownership information from certificate content while ensuring that both remain permanently verifiable through blockchain and IPFS technologies.

4.6 Wallet Integration

The proposed system adopts a non-custodial wallet model in which students retain full control of their blockchain identities. Instead of generating wallets on behalf of users, the platform integrates with MetaMask, a browser-based Ethereum wallet extension.

When a student accesses the portal, the application requests authorization through the browser's injected Ethereum provider. Upon approval, the selected wallet address is retrieved and displayed within the user interface. The captured address is then submitted to the backend and associated with the student's academic profile. The wallet registration workflow is shown below:



Only public wallet addresses are stored within the database. Private keys never leave the student's MetaMask wallet and remain under the student's exclusive control. After wallet registration, the stored address becomes the destination for NFT certificate issuance. During the graduation process, the backend retrieves the student's registered wallet address and invokes the smart contract minting function through the university's administrative blockchain account. The newly created NFT certificate is therefore transferred directly to the student's wallet. This architecture enables decentralized certificate ownership while eliminating the need for the institution to manage or store user private keys.

4.7 Smart Contract Security Analysis

4.7.1 Unauthorized Minting Prevention

1. Threat Model and Attack Vector

In a decentralized academic certification ecosystem, the integrity of the entire platform relies heavily on the authenticity of the minting process [25]. Because a minted Non-Fungible Token (NFT) represents an official, legally binding university degree [34], the `mintCertificate(...)` function is the most critical and highly targeted entry point in the smart contract architecture [13].

An external malicious actor (attacker) or an unprivileged user could attempt to execute a direct contract transaction calling the `mintCertificate(...)` function by targeting the deployed contract address on the network [33]. The Attacker's Objective:

- To bypass institutional verification gates and self-mint an unauthorized diploma [32]
- To dilute the trust of the academic registry by injecting fraudulent credentials into the ledger [27]

Without rigorous access control mechanisms embedded directly into the smart contract state machine [20], any network entity possessing an EVM-compatible wallet address could broadcast a transaction containing arbitrary student metadata arrays, forcing the blockchain to anchor

fake certificates permanently [30].

2. Solution: Architectural Implementation of onlyOwner

To neutralize this attack vector, the smart contract utilizes an explicit access control pattern inherited from OpenZeppelin's standardized security libraries [25]. By binding the onlyOwner modifier directly to the execution signature of the minting function, the platform establishes an immutable cryptographic boundary [20]. The onlyOwner mechanism operates on a simple but absolute identity evaluation principle [20]:

```
function mintCertificate(  
    address student,  
    string memory ipfsHash  
) public onlyOwner returns (uint256) {  
    // Minting execution logic only proceeds if msg.sender == owner  
    return _executeMint(student, ipfsHash);  
}
```

During the contract compilation and initialization phase handled within the development suite [35], the account address used to deploy the contract (the University Administrator's primary deployment key) is permanently recorded in the contract's storage state as the `_owner` [14]. When any transaction triggers `mintCertificate(...)`, the EVM intercepts the instruction and executes an internal validation rule: `require(msg.sender == owner, "Ownable: caller is not the owner")`; [20][16]. If the public key hashing the transaction signature (`msg.sender`) does not match the stored administrator address, the condition evaluates to false, immediately breaking execution [13][14].

3. Result and State Reversion

When an unauthorized entity initiates a call to `mintCertificate(...)`, the blockchain engine enforces an atomic rollback [15].

- **State Preservation:** The transaction fails completely [2]. No new Token ID is generated, no IPFS metadata is bound [21], and no state variables inside the MongoDB database mirror or the local Hardhat node are modified [35].
- **Gas Consumption Penalties:** The attacker's transaction is forcefully marked as Status: Fail (Reverted) [14]. All state modifications are completely rolled back as if the transaction never occurred, and the remaining execution gas is consumed, adding an economic penalty for the malicious actor [14][15].
- **Error Logs:** The system emits a clear execution exception, typically logging Error: VM Exception while processing transaction: reverted with reason string 'Ownable: caller is not the owner' [35].

```

function mintCertificate(
    address student,
    string memory studentId,
    string memory studentName,
    string memory degree,
    string memory major,
    string memory ipfsHash,
    string memory fileHash
) public onlyOwner {
    require(hashToTokenId[fileHash] == 0, "A certificate with this file hash already exists");

    uint256 tokenId = ++nextTokenId;

    _safeMint(student, tokenId);

    certificates[tokenId] = Certificate(
        studentId,
        studentName,
        degree,
        major,
        ipfsHash,
        block.timestamp,
        false,
        0,
        "",
        fileHash
    );

    hashToTokenId[fileHash] = tokenId;
}

```

Figure 25: minCertificate Function - Smart Contract Access Control Protection

```

=====
khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert/blockchain$ npx hardhat run scripts/test-revert.ts --network localhost
  ◦ injected env (1) from .env // tip: ✎ override existing { override: true }
  ◦ injected env (0) from .env // tip: ✎ enable debugging { debug: true }
Attacker Address: 0x78997978c51812dc3A010C7d01b50e0d1dc79C8
Sending explicit transaction to trigger onlyOwner reversion...
EVM Reversion Catch: Ownable: caller is not the owner
=====
khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert/blockchain$

```

Figure 26: EVM Transaction Reversion Terminal Log

4.7.2 Duplicate Certificate Prevention

1. Threat Model and Collision Attack Vector

The value of an academic degree on a public ledger stems entirely from its uniqueness and non-fungibility [34]. In a flawed implementation, a compromised academic administrator or an attacker exploiting an edge case could attempt a Reissue or Duplicate Mint Attack [13]. The Attacker's Objective:

- To reissue an identical certificate payload twice, assigning the duplicate credential to a different student address [32].
- To artificially inflate graduation metrics or counterfeit credentials by copying a legitimate student's IPFS metadata profile [27].

Because metadata hashes (such as IPFS CIDs) are stored as public records on-chain, any network participant can read a valid certificate payload [21]. If the smart contract state machine only relies on an incremental token counter (e.g., `_nextTokenId++`) without anchoring the cryptographic hash of the file to a uniqueness check, the EVM will happily process the transaction [14]. This would create two distinct Token IDs pointing to the exact same academic file hash,

permanently damaging the ledger's authority [25].

2. Solution: Architectural Implementation of State Uniqueness Checks

To permanently eliminate this attack vector, the smart contract registers every single file hash into a global lookup mapping state variable during execution: `mapping(string => uint256)` `private hashToTokenId`; [14]. Before executing the underlying ERC-721 minting routine, the contract evaluates the incoming payload using a strict validation constraint [20]:

```
// Cryptographic uniqueness verification gate
require(
    hashToTokenId[fileHash] == 0,
    "A certificate with this file hash already exists"
);
```

When a certificate is minted for the first time, its IPFS file hash does not exist in the mapping, returning the default storage value of 0 [14]. The condition passes, and the contract writes the new Token ID into that slot. If anyone attempts to pass the exact same string (`fileHash`) again, the mapping evaluates to an integer greater than zero [18]. The `require` statement instantly catches the collision, short-circuits the stack execution, and throws a clear exception string [13].

3. Result and State Reversion

This structural pattern guarantees that a duplicate NFT certificate is cryptographically impossible to generate within the ecosystem [36].

- **EVM Reversion:** The transaction fails atomically [15]. No gas is wasted on computing secondary storage changes, and the network immediately rejects the block [37].
- **Database Synchronization Integrity:** Because the transaction reverts at the protocol layer, the backend listener fails to capture a Transfer event [24]. This prevents any corrupted data state from being mirrored to your off-chain MongoDB instance or local cache pipelines [35].
- **Immutable Security Proof:** The system achieves a strict 1:1 invariant relationship between a physical academic achievement and its digital blockchain anchor [29].

```

ert/blockchain$ npx hardhat test
khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert/blockchain$ npx h
ardhat test
  injected env (1) from .env // tip: ✦ secrets for agents [www.dotenvx.com]

  AcademicCertificate Security Validations
[STAGE 1] Minting initial valid certificate to Ledger...
  -> Target Student: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
  -> Payload Hash: 0x7465737466696c6568617368303030303030303030303030303032
  Result: Transaction Successful. State committed to EVM ledger.

[STAGE 2] Submitting unauthorized duplicate payload block...
  -> Target Student: 0x3C44CdD86a900fa2b585dd299e03d12FA4293BC (Malicious or duplicate ent
ry)
  -> Collision Hash: 0x7465737466696c6568617368303030303030303030303030303032
  Result: Transaction Rejected dynamically by State Validation Guard.
    ✓ Should prevent duplicate minting of the same certificate hash (332ms)

  1 passing (334ms)
khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert/blockchain$

```

Figure 27: Hardhat Test Terminal Catching Duplicate Hash Rejection

4.7.3 Soulbound Enforcement

1. The Vulnerability of Standard ERC-721 Tokenization in Academic Credentials

The standard tokenization framework defined by the Ethereum Improvement Proposal (EIP-721) was originally engineered to facilitate the friction-free exchange of digital assets, collectible items, and financial instruments across decentralized networks [34]. To support these secondary liquid markets, the default OpenZeppelin ERC-721 baseline implementation exposes global, publicly accessible state-changing methods, specifically:

- `transferFrom(address from, address to, uint256 tokenId)`
- `safeTransferFrom(address from, address to, uint256 tokenId, bytes data)`

While these native features are ideal for tradeable assets, they present a fundamental architectural vulnerability when directly applied to institutional certification ecosystems [25]. Academic degrees, professional licenses, and graduation certificates are intrinsically tied to an individual's unique identity, historical achievement, and verified merit [27]. If a university issues a diploma as a vanilla ERC-721 token, the recipient (Student A) retains full cryptographic custody over that asset's state transitions [13].

Consequently, Student A could execute an unauthorized secondary transfer, sending the token to Student B via a direct peer-to-peer transaction or listing it on a public decentralized marketplace (such as OpenSea) for financial gain. This loophole would allow bad actors to purchase, sell, or illicitly inherit authentic academic credentials, entirely breaking the trust assumptions, systemic authority, and verification integrity of the decentralized ledger [32].

2. Architectural Design of Soulbound Tokens (SBTs)

To neutralize this exploit vector and enforce absolute immutability over credential ownership, the system design mutates the standard token layout into a Soulbound Token (SBT) in-

frastructure [37]. A Soulbound Token is conceptually defined as a non-transferable, identity-anchored digital asset bound permanently to a specific ledger account address (the "Soul") from the exact millisecond of its initial generation [29].

The core technical challenge in creating an SBT within an EVM runtime environment involves overriding the asset's internal state-transition rules without breaking downstream read-only compatibility with mainstream Web3 block explorers (such as Etherscan) or decentralized indexing protocols (such as The Graph) [18]. To achieve this, the underlying smart contract overrides the native internal hook mechanics of the ERC-721 standard library, cutting off all transfer execution logic at the protocol layer before any storage changes can be written to the block [14].

3. OpenZeppelin v4 Smart Contract Override Implementation

In OpenZeppelin v4 contract configurations, all public-facing transfer functions inside the standard library route their core state updates through an internal execution lifecycle hook called `_beforeTokenTransfer`. By intentionally overriding this hook method and placing a conditional validation gate inside its block, we intercept every single asset transfer attempt made across the network [20].

```
function _beforeTokenTransfer(
    address from,
    address to,
    uint256 firstTokenId,
    uint256 batchSize
) internal virtual override {
    super._beforeTokenTransfer(from, to, firstTokenId, batchSize);

    // Enforce Soulbound Invariant: Revert if the transaction is a transfer
    require(
        from == address(0),
        "AcademicSBT: Transfer restriction enforced - Credentials are non-transferable"
    );
}
```

Let's explain the code:

- **The Minting Exception:** When the university administrator triggers `mintCertificate()`, the token transits from the non-existent state (`address(0)`) to the target student's wallet. The contract evaluates `from == address(0)`, which returns true. The validation check passes cleanly, and the initialization function completes successfully [14].
- **The Transfer Block:** If any actor later attempts to call `transferFrom()` or `safeTransferFrom()`, the internal state engine routes the data stream into this overridden hook before updating the token's registry mappings [18]. The parameter `from` evaluates to the student's current wallet address (which is not `address(0)`). The `require` constraint instantly fails, intercepts the call stack, throws an exception string, and triggers an absolute state reversion [24].

4. Structural Data Flow and Rejection Mechanics

The diagram below maps the complete lifecycle of a blocked transfer transaction under the OpenZeppelin v4 architecture. When a user or a third-party script triggers a transfer payload, the transaction is pushed to the local EVM node where the runtime memory space isolates the execution failure, cancels the transaction pipeline, and preserves the original ownership state perfectly.

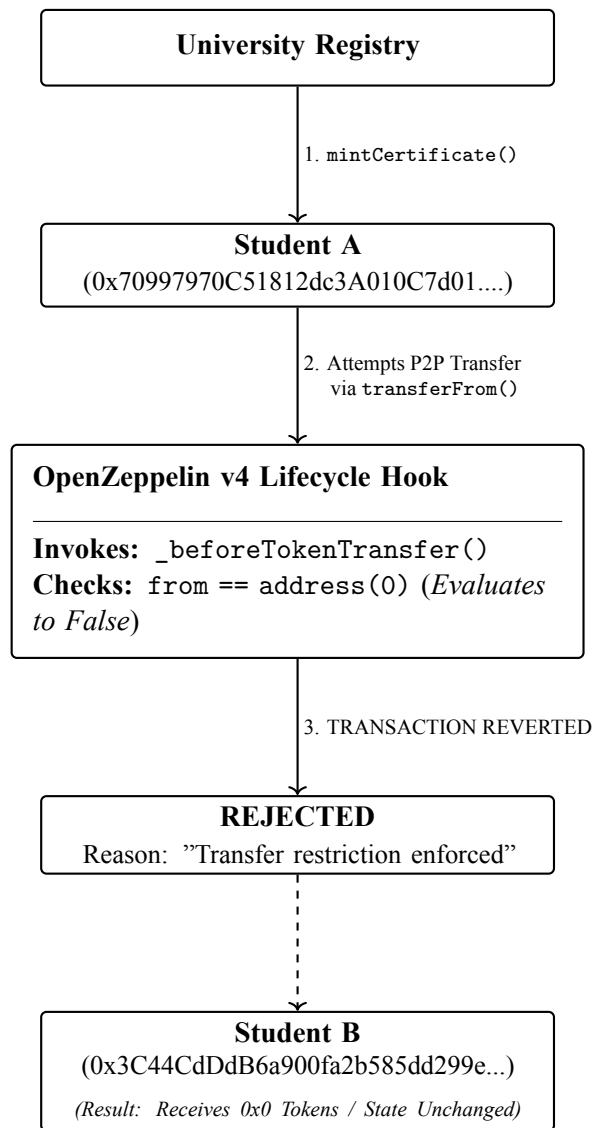


Figure 28: Lifecycle of a blocked transfer transaction

4.7.4 Hash Tampering Protection

1. Cryptographic Hashing and Data Integrity

To protect certificate integrity, the proposed system uses the SHA-256 cryptographic hash algorithm [34]. A hash function converts an input file into a fixed-length 256-bit value, commonly represented as a 64-character hexadecimal string [20].

In this system, the generated hash acts as a digital fingerprint of the certificate. Even a very small modification to the document, such as changing a single character or punctuation mark,

produces a completely different hash value. This property is known as the *Avalanche Effect* [25]. In a secure hashing algorithm, a minute modification to the input dataset—even a single bit flip or changing a single punctuation mark inside an academic transcript—will cause a cascade of changes throughout the calculation steps. As a result, the final output hex string changes completely and unpredictably, bearing no statistical correlation to the original hash string [18]. The mathematical properties of this protection architecture include:

- **Pre-image Resistance:** Given a target hash H , it is computationally impossible to reconstruct the original certificate file contents x such that $\text{SHA-256}(x) = H$ [29].
- **Collision Resistance:** It is extremely difficult to find two distinct certificate files that generate the exact same output hash signature [14].
- **Deterministic Execution:** The identical certificate file will always yield the exact same 64-character hex signature, allowing for decentralized verification without a central authority [37].

2. Certificate Tampering Scenario

In traditional digital credential systems, an attacker or a dishonest student can easily manipulate PDF documents using standard editing software [27]. For example, a student might modify their GPA from 2.8 to 3.8, or alter the text inside the major field from Computer Science to Artificial Intelligence [13]. Since standard PDF files do not natively contain an immutable tamper-evident validation layer, these changes are hard to detect unless a verifier manually contacts the issuing university registry [32]. Within our decentralized platform, when a university issues a degree, the system executes a dual-anchoring sequence:

- The complete structured student profile metadata is written to the InterPlanetary File System (IPFS) [21].
- The raw, original binary array of the physical certificate PDF file (Certificate.pdf) is hashed locally inside the client browser interface using SHA-256 to generate a unique digital fingerprint, which is passed directly to the smart contract's fileHash state variable [14].

If a student attempts to upload a modified PDF file to a validation portal, the verification engine dynamically computes the SHA-256 signature of the uploaded file on-the-fly and compares it against the immutable hashToTokenId tracking mapping recorded on the blockchain ledger [18].

3. Demonstration: The Impact of Single-Character Manipulation

To demonstrate the sensitivity of the system's tamper-detection mechanism, The below table tracks the output values generated when a standard degree certificate payload undergoes minor unauthorized modifications.

Scenario	Input Target File (Certificate.pdf)	Computed SHA-256 Hexadecimal Hash Signature	System Verifi- cation Status
Baseline	Name:NguyenThiHoa ,Major:SoftwareEn gineering,.....,G PA:2.3	8b5a1f6c4d2e...3e7 f9a0b1c2	VALID
Tamper Attempt 1	Name:NguyenThiHoa ,Major:SoftwareEn gineering,.....,G PA:3.2	4f7e2a1b9c0d...a6b 7c8d9e0f	REJECTED (Hash Mis- match)
Tamper Attempt 2	Name:NguyenThiHoa ,Major:SoftwareEn gineerinG,.....,G PA:3.8	c9a1b2c3d4e5...f6a 7b8c9d0e	REJECTED (Hash Mis- match)

Table 7: SHA-256 Output Variance Tracing via Minor Payload Alterations

As shown in the evaluation metrics above, changing a single decimal point in the GPA string (from 2.3 to 3.2) or changing a single lowercase character to uppercase (Engineering to EngineerinG) creates an entirely different cryptographic signature [25]. The new signature bears no structural resemblance to the baseline hash value registered on-chain.

4. Dynamic Verification Logic and Rejection Mechanics

When a third-party verifier (such as a recruiter or a corporate employer) drops a certificate PDF onto the verification portal interface, the system executes the following verification pipeline [35]:

$$\text{Verification Status} = \begin{cases} \text{Success,} & \text{if hashToTokenId[SHA-256(Uploaded File)]} > 0 \\ \text{Failure,} & \text{if hashToTokenId[SHA-256(Uploaded File)]} = 0 \end{cases}$$

If any data has been tampered with, the lookup mapping searches for the altered hash variant string (XYZ999...) instead of the true record string (ABCD123...) [14]. Because the altered hash string has never been initialized by the onlyOwner institutional minting routine, the EVM lookup evaluates to its default uninitialized integer value: 0 [18]. The client web application captures this null index response, flags the asset as a forged credential, and updates the user interface with a clear warning notification [35]. This design pattern completely eliminates credential forgery. An attacker cannot alter a single byte of their academic document without instantly breaking the underlying cryptographic proof, ensuring absolute security across the ecosystem [29].

4.7.5 Wallet Ownership Security

1. The Paradigm of Self-Sovereign Identity and Ledger Ownership

In traditional centralized enterprise applications, user accounts, data profiles, and access permissions are managed by a central database administrator using standard authentication systems (such as username/password pairs or OAuth tokens) [35]. Under that legacy framework, users do not own their identity data; they are merely granted access to a database row that can be modified, revoked, or compromised by a central authority or database breach.

Our decentralized platform shifts this paradigm to a Self-Sovereign Identity (SSI) and cryptographic ownership model [27]. Within this architecture, an academic degree or certificate is not an arbitrary row in an institutional database. Instead, it is an independent digital asset (a Soulbound NFT) permanently mapped to a specific, unique cryptographic wallet address on the blockchain ledger [34]. The ownership of this asset is strictly governed by asymmetric cryptography, where the student's wallet address acts as a public identifier, and access control is enforced entirely by the corresponding mathematical private key [29].

2. The Cryptographic Asymmetry: Public Addresses vs. Private Keys

The security of an individual's academic credential portfolio relies on the mathematical relationship between two keys generated via the Elliptic Curve Digital Signature Algorithm (ECDSA), specifically the secp256k1 curve standard used by Ethereum-compatible networks [18]:

1. The Public Key / Wallet Address (A_{pub}): This is a 20-byte (160-bit) hexadecimal string derived from the student's public key (e.g., 0x7099...79C8). It acts as a public identity anchor. The smart contract maps the issued Token ID directly to this address inside its internal ownership registry: `mapping(uint256 => address) private _owners;` [14].
2. The Private Key (k_{priv}): This is a secret 256-bit randomly generated integer that is kept entirely confidential by the student (usually managed securely inside an isolated hardware module or a non-custodial software wallet like MetaMask) [25].

Because the token is structurally bound to the wallet address, only the holder of the matching private key can interact with or prove ownership of that specific academic degree [13]. The private key is mathematically required to generate a valid cryptographic signature (σ) for any outbound transaction or off-chain verification challenge [20]. Without access to this private key, it is computationally impossible for an outside attacker to forge a signature or impersonate the credential holder, ensuring absolute security across the ecosystem [29].

3. Smart Contract Access Control and Institutional Isolation

To protect the student from unauthorized delegation, credential theft, or accidental listings on secondary marketplaces, the `AcademicCertificate` contract explicitly intercepts token custody state changes using internal lifecycle overrides from the base OpenZeppelin ERC-721 library [37].

Rather than solely masking user-facing methods, the contract establishes a foundational execution constraint at the core token transfer pipeline. This architectural strategy guarantees that any high-level transfer invocation instantly reverts unless it is part of the initial minting routine:

```
function _beforeTokenTransfer(
    address from,
    address to,
    uint256 firstTokenId,
    uint256 batchSize
) internal virtual override {
    super._beforeTokenTransfer(from, to, firstTokenId, batchSize);

    require(
        from == address(0),
        "AcademicSBT: Transfer restriction enforced - Credentials are non-transferable"
    );
}

function approve(address, uint256) public pure override {
    revert("Soulbound: No approvals allowed");
}

function setApprovalForAll(address, bool) public pure override {
    revert("Soulbound: No approvals allowed");
}
```

- **Lifecycle Hook Interception:** Intercepting the internal `_beforeTokenTransfer` loop creates a centralized validation point. Because token minting operations route through this identical sequence, the logical assertion evaluates the source address parameter. If the `from` state is not the zero-address (`address(0)`), the system recognizes the transaction as an impermissible secondary market move or peer-to-peer exchange and forcefully terminates execution [18].
- **Exploitation Surface Mitigation:** While the internal hook stops actual token movement, the public interface explicitly overrides `approve()` and `setApprovalForAll()`. Uniquely, these are declared with the `pure` visibility modifier since their execution blocks immediately call `revert()` without reading or writing to the underlying contract state. This proactively neutralizes third-party market listings before any transactional payloads hit the state machine [18].
- **Deterministic Collision Resistance:** To maintain rigorous document validation standards, the minting mechanism tracks document uniqueness using a strict state mapping:
`mapping(string=>uint256) public hashToTokenId;`
 Before a token is securely allocated, the contract evaluates whether `hashToTokenId[fileHash] == 0`. This prevents an administrative error or threat actor from duplicate-minting an identical cryptographic file signature onto multiple distinct token IDs [18].
- **Institutional Isolation:** Furthermore, administrative controls are tightly sandboxed using the standard OpenZeppelin `onlyOwner` modifier [13]. While the university registry maintains exclusive authorization to invoke `mintCertificate()` and `revokeCertificate()`, it holds no operational control or recovery vector to manipulate or alter a certificate once it is cryptographically anchored to a student's public address [14].

4. Structural Authentication and Ownership Verification Flow

The diagram below maps how the system verifies wallet ownership. When a student attempts to access their profile or present their credentials to a verification portal, the application uses an off-chain cryptographic challenge-response routine (EIP-712) to verify private key ownership without exposing the key itself.

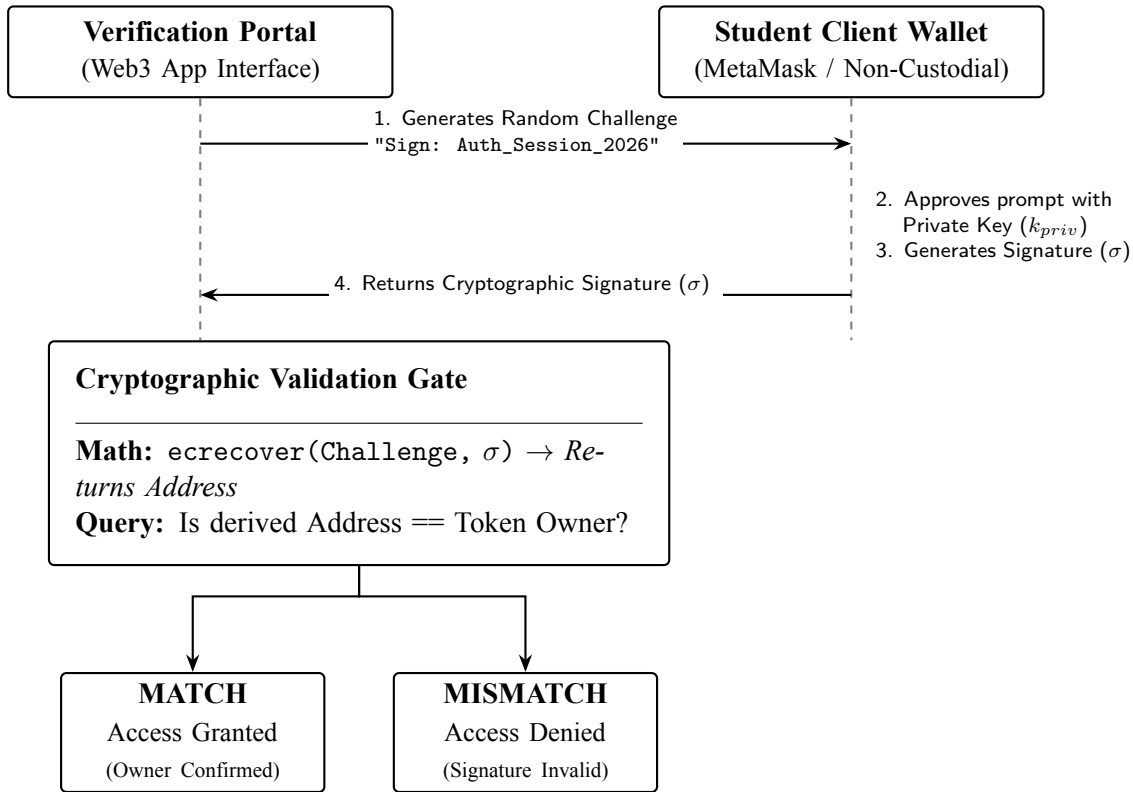


Figure 29: Off-Chain Verification Mechanism via Asymmetric ECDSA Challenge-Response Handshake.

5. Deterministic Security Results

This architectural layout guarantees several deterministic security outcomes for the student's data portfolio:

- **Immunity to Database Hacks:** Because the credentials exist on a decentralized ledger rather than a centralized server, a security breach at the university's IT infrastructure cannot result in the theft, modification, or leakage of already-issued student certificate assets [32].
- **Absolute Asset Custody:** The student maintains absolute lifetime custody of their verified achievements [27]. No university administrator, corporate entity, or malicious network participant can manipulate the underlying state mappings without possessing the student's unique private key [18].
- **Phishing and Front-Running Resistance:** Since approvals and peer-to-peer transfers are entirely disabled at the internal contract layer, students are completely immune to common Web3 phishing attacks (such as "blind signing" a malicious marketplace approval

transaction), ensuring that their academic portfolio remains secure even if they interact with an untrusted decentralized application [25].

4.8 Deployment Process

To validate the operational capabilities, structural safety gates, and data persistence pipelines of the decentralized academic registry, a comprehensive multi-tier local deployment was carried out. The execution infrastructure leverages an isolated, high-performance local Ethereum virtual machine (EVM) node powered by the Hardhat Development Framework running on an Ubuntu environment.

By prioritizing a fully simulated, low-latency local node over a live public testnet (such as Sepolia), the development pipeline minimizes transaction confirmation latency, eliminates the volatility of testnet gas pricing, and ensures a deterministic sandbox environment for testing edge-case boundary conditions, such as malicious duplicate data submission and cross-tier API state syncing.

4.8.1 Hardhat Local Deployment

The foundational phase of the deployment lifecycle requires bootstrapping a local standalone Ethereum node instance. This virtual environment acts as an isolated, single-node private blockchain ledger running in the background of the development machine.

Execution Process

To spin up the internal EVM execution environment, a clean terminal instance is instantiated inside the core workspace directory, and the Hardhat node task runner is executed via the node package executor:

```
cd ~/Desktop/blockchain/nft-cert/backend  
npx hardhat node
```

Technical Functionality and Allocation

Upon execution, the Hardhat engine instantiates an in-memory blockchain node listening on the local host loopback adapter at `http://127.0.0.1:8545`. The node dynamically pre-generates 20 distinct cryptographic developer accounts. Each account is automatically provisioned with a virtual balance of 10,000 ETH to facilitate gas payment calculations across subsequent transaction blocks.

Crucially, the initialization sequence exposes the raw unencrypted Private Keys and matching Public Hex Addresses of these accounts to the console, allowing the first account (Account #0) to be instantly mapped as the central university registry administrator (`onlyOwner`), while secondary accounts are reserved for student identity mapping and malicious signature testing.

```

• khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert$ cd blockchain/
○ khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert/blockchain$ npx hardhat node
  ↳ injected env (1) from .env // tip: ✦ encrypted .env [www.dotenvx.com]
  Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts
=====

WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.

Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80

Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Private Key: 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d

Account #2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000 ETH)
Private Key: 0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a

Account #3: 0x90F79b66f6B2c4f80365E785982E1f01E93b906 (10000 ETH)
Private Key: 0x7c852118294e51e653712a81e0500f419141751be58f605c371e15141b007a6

Account #4: 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65 (10000 ETH)
Private Key: 0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a

Account #5: 0x9965507D1a55bc2695C58ba16F837d819B0A4dc (10000 ETH)
Private Key: 0x8b3a350cf5c34c9194ca85829a2df0ec3153be0318b5e2d3348e872092edffba

Account #6: 0x976EA74026E726554d8657fA54763abd0C3a0aa9 (10000 ETH)
Private Key: 0x92db14e403b3dfe3df233f83d3fa0d7096f21ca9b0d6d6b8d88b2b4ec1564e

Account #7: 0x14dC79964da2C08b23698B3D3cc7Ca32193d9955 (10000 ETH)
Private Key: 0x4bbbf85ce3377467afe5d46f804f221813b2bb87f24d81f60f1fcd9f7cbf4356

```

Figure 30: Initialization Logs of the Local Hardhat EVM Node

4.8.2 Smart Contract Compilation

Before human-readable Solidity programming patterns can be recognized or processed by the local Hardhat EVM node, the source file code must be transformed into low-level machine components.

```
npx hardhat compile
```

The Hardhat compilation suite initializes the Solc (Solidity Compiler) version 0.8.19 engine, running a parsing pass over `AcademicCertificate.sol` along with its inherited dependencies from the OpenZeppelin v4 framework library (`ERC721.sol`, `Ownable.sol`). The parsing compiler breaks the source code down into two discrete, high-value asset structures stored inside the local `/artifacts` system folder:

- **The Application Binary Interface (ABI):** A comprehensive JSON-formatted schema mapping file that explicitly documents every function signature, argument array, mutability state, and event tracking structure exposed by the contract. The frontend and backend tiers parse this JSON file to learn exactly how to format data packets before sending them across the JSON-RPC highway.
- **The EVM Bytecode:** A highly optimized stream of raw hexadecimal operation instructions (opcodes). This bytecode represents the physical application blueprint that is permanently written into the immutable storage state of the blockchain during deployment.

```

41  ...
42  ---> NOTE: copy deploy address from this to put in the .env file of /backend folder
43
44
45  ### 2. Backend start
46
47  open new terminal
48

```

```

• khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert/blockchain$ cd blockchain/
• khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert/blockchain$ npx hardhat clean
  ◊ injected env (1) from .env // tip: ~ auth for agents [www.vestauth.com]
• khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert/blockchain$ npx hardhat compile
  ◊ injected env (1) from .env // tip: ~ auth for agents [www.vestauth.com]
Generating typings for: 13 artifacts in dir: typechain-types for target: ethers-v6
Successfully generated 38 typings!
Compiled 13 Solidity files successfully (evm target: paris).
❖ khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert/blockchain$

```

Figure 31: Hardhat Compiler Output Confirmation Panel

4.8.3 Smart Contract Ledger Deployment

The automated deployment tracking script is executed using the `--networklocalhost` flag, instructing Hardhat to bypass its transient in-memory node and target the active persistent loop-back server established in Step 1:

```
npx hardhat run scripts/deploy.ts --network localhost
```

- **The Deployment Transaction:** The script `deploy.ts` instantiates an automated contract factory instance utilizing the account signature of Account #0. It frames a specialized transaction payload where the destination (`to` field) is set to a null state (`0x000...000`), and the data payload field contains the entire compiled contract bytecode.
- **State Storage Anchor:** The local node processes the block, calculates the required creation gas overhead, and computes a deterministic cryptographic address for the contract based on the deployer's public address and transaction nonce.
- **Environmental Mapping Sync:** The console prints the live contract address. As outlined in the architecture requirements, this generated address string must be manually copied and updated inside the `backend/.env` file environment variables array (`CONTRACT_ADDRESS=0x5FbDB2315678afecb367f032d93F642f64180aa3`). This update ensures the backend Express server knows exactly which address to hit on-chain when a recruiter requests record validation.

```

Compiled 13 Solidity files successfully (evm target: paris).
khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert/blockchain$ npx hardhat run scripts/deploy.ts --
network localhost
  injected env (1) from .env // tip: ✖ enable debugging { debug: true }
  injected env (0) from .env // tip: ✖ override existing { override: true }
Starting deployment of AcademicCertificate contract...

=====
AcademicCertificate successfully deployed!
Contract Address: 0x5FbDB2315678afecb367f032d93F642f64180aa3
=====
khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert/blockchain$

```

Figure 32: Terminal Receipt of Successful Contract Ledger Deployment

4.8.4 Environment Syncing and System State Configuration

With the smart contract live on the local node network, the next phase bridges the blockchain layer with the off-chain backend architecture. The deployment process utilizes environment variables to securely store the contract's address, ensuring the application code remains completely modular.

1. Environmental Address Mapping Synchronization

The operational configuration variables are structured across four specific functional layers within the backend runtime space:

- **EVM Network Provider Settings:** Links the server directly to the active Hardhat loop-back adapter engine (`RPC_URL=http://127.0.0.1:8545`) and declares the dynamic on-chain deployment target destination (`CONTRACT_ADDRESS=0x5FbDB2315678afecb367f032d93F642f64180aa3`). Administrative authorization is simulated using the node's initial pre-allocated structural private key (`PRIVATE_KEY=0xac0974bec3...`).
- **Decentralized Cloud Storage Pipeline:** Establishes connection criteria to the Pinata IPFS protocol gateway (`PINATA_URL`) using an authenticated JSON Web Token bearer signature string (`PINATA_JWT`). This allows the server to programmatically pin immutable JSON metadata assets directly to the decentralized peer-to-peer storage fabric.
- **Persistent Database Tier:** Configures connection criteria to a remote cloud storage infrastructure using an authenticated MongoDB Atlas shared cluster link endpoint (`MONGO_URI`).
- **Automated Notification Infrastructure:** Integrates transaction log notifications directly into an external Simple Mail Transfer Protocol gateway routing network (`EMAIL_HOST=smtp.gmail.com`). This service automatically coordinates asynchronous notification dispatches to graduates when a block transaction completes successfully.

```

PORT=5000
FE_URL=http://localhost:5173
MONGO_URI=mongodb+srv://huynhbaokhuyen123_db_user:[HIDDEN]@cluster0.30cfhmt.
mongodb.net/
CONTRACT_ADDRESS=0x5FbDB2315678afecb367f032d93F642f64180aa3
RPC_URL=http://127.0.0.1:8545
PINATA_JWT=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
PRIVATE_KEY=0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587

```

```
EMAIL_USER=huynhbaokhuyen123@gmail.com
PINATA_URL=https://gateway.pinata.cloud/ipfs
GROQ_API_KEY=gsk_yPlAcAGCutxNm4hQVRc2...
```

2. Backend Application Service Boot Sequence

With the decoupled environment profiles locked in, an active process thread is instantiated inside the /backend development path to mount the Node.js TypeScript compilation engine:

```
cd backend
npm run dev
```

Upon bootstrapping, the server parses the system parameters array to safely bind its application components. The engine initializes a persistent database listener loop to the MongoDB cloud cluster, connects an internal instance of the ethers.JsonRpcProvider hook directly to your Hardhat engine, and opens a public HTTP network socket listener on port 5000.



```
khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert/backend$ npm run dev
> backend@1.0.0 dev
> nodemon index.ts

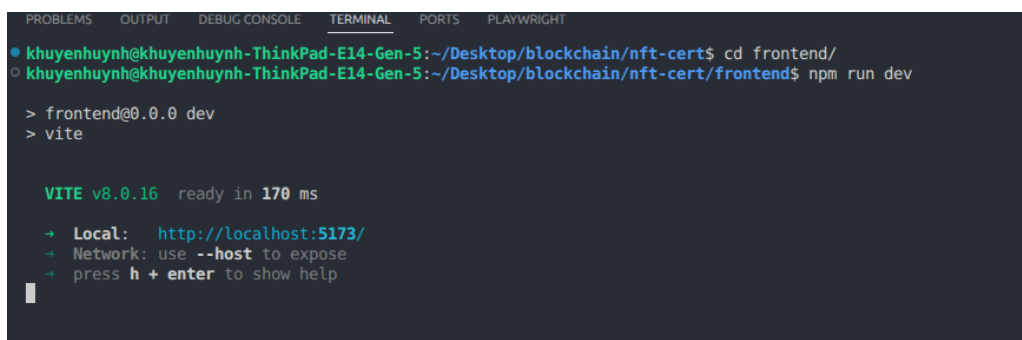
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: ts,json
[nodemon] starting `ts-node index.ts`
◊ injected env (13) from .env // tip: ✖ enable debugging { debug: true }
◊ injected env (0) from .env // tip: ✖ encrypted .env [www.dotenvx.com]
◊ injected env (0) from .env // tip: ✖ enable debugging { debug: true }
🟢 Successfully connected to Local MongoDB
🟢 Backend server is running locally on http://localhost:5000
```

Figure 33: Express REST API Server Initialization Log Sequence

4.8.5 Frontend side for user interface

We also need an UI for interact, so this section for how to run the frontend.

```
cd frontend
npm run dev
```



```
khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert$ cd frontend/
khuyenhuynh@khuyenhuynh-ThinkPad-E14-Gen-5:~/Desktop/blockchain/nft-cert/frontend$ npm run dev
> frontend@0.0.0 dev
> vite

VITE v8.0.16 ready in 170 ms
➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
➔ press h + enter to show help
```

Figure 34: Run Front-end

5 AI-ASSISTED NFT CERTIFICATE VERIFICATION

5.1 AI Asisstant Concept

Although blockchain technology provides a secure and immutable mechanism for certificate verification, the underlying technical information may be difficult for non-technical users to understand. Verification results often contain transaction hashes, wallet addresses, token identifiers, and IPFS references that are unfamiliar to recruiters, employers, or university administrators.

To improve usability, this system introduces an AI-Assisted Certificate Verification Module. The purpose of this module is not to perform blockchain verification independently, but rather to translate verified blockchain data into clear and understandable natural language explanations.

The AI assistant acts as an interpretation layer between the blockchain verification system and end users. After certificate information has been validated by the application, the assistant analyzes the verified data and provides human-readable explanations regarding:

- Certificate authenticity status.
- NFT ownership information.
- Blockchain transaction records.
- IPFS metadata storage references.
- Academic certificate details.
- Certificate lifecycle status.

This approach allows users to interact with blockchain verification results through conversational questions rather than manually interpreting technical records.

5.2 AI Technology Stack

The AI verification module utilizes a cloud-hosted Large Language Model (LLM) architecture integrated into the existing web application.

Component	Technology	Purpose
Frontend Interface	React + TypeScript	Provides AI chat interface and verification dashboard
Backend API Layer	Node.js + Express	Handles AI requests and response streaming
AI Service Provider	Groq Cloud API	Provides access to hosted LLM models
Language Model	Llama 3.3 70B Versatile	Generates natural language explanations

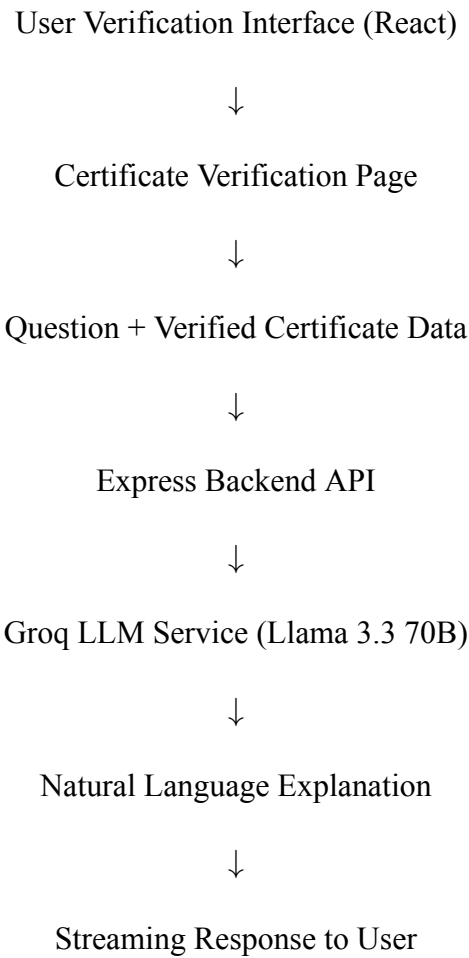
Certificate Source	Data	MongoDB + Blockchain Records	Supplies verified certificate information
Streaming Engine		Server-Sent Text Streaming	Delivers AI responses incrementally

Table 8: AI Tech Stack

Unlike traditional AI agent systems that directly query external services, the proposed architecture relies on pre-verified certificate information supplied by the application. This design reduces hallucination risk and improves response reliability.

5.3 System Architecture and Workflow

The AI verification workflow follows the architecture shown below:



The workflow consists of the following stages:

5.3.1 Step 1 – Certificate Verification

The user accesses a verified NFT certificate through the platform. The application retrieves certificate information from the database and blockchain records, including:

- Student name
- Token ID
- Degree
- Major
- Wallet address
- IPFS hash
- Certificate lifecycle status

5.3.2 Step 2 – User Question Submission

The user enters a natural language question through the AI assistant interface. Examples include:

- "Is this certificate authentic?"
- "Who owns this NFT certificate?"
- "What does the IPFS hash represent?"
- "Has this certificate been revoked?"

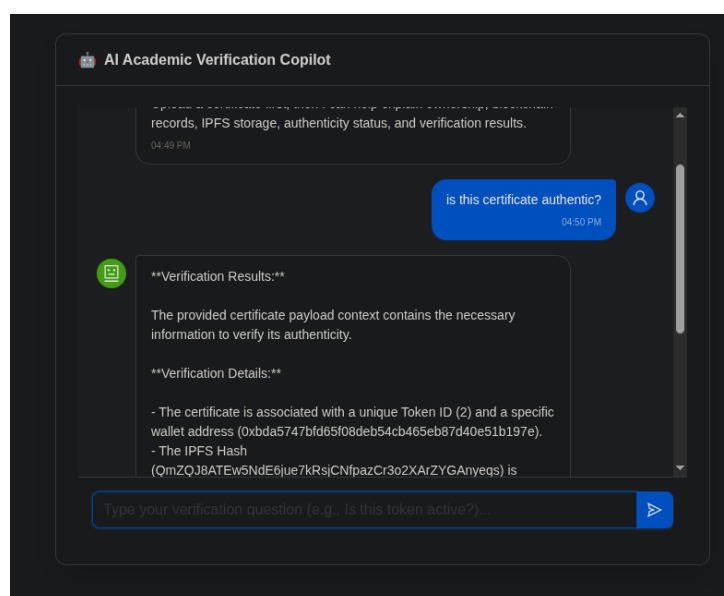


Figure 35: AI Assistant - Question: *Is this certificate authentic?*

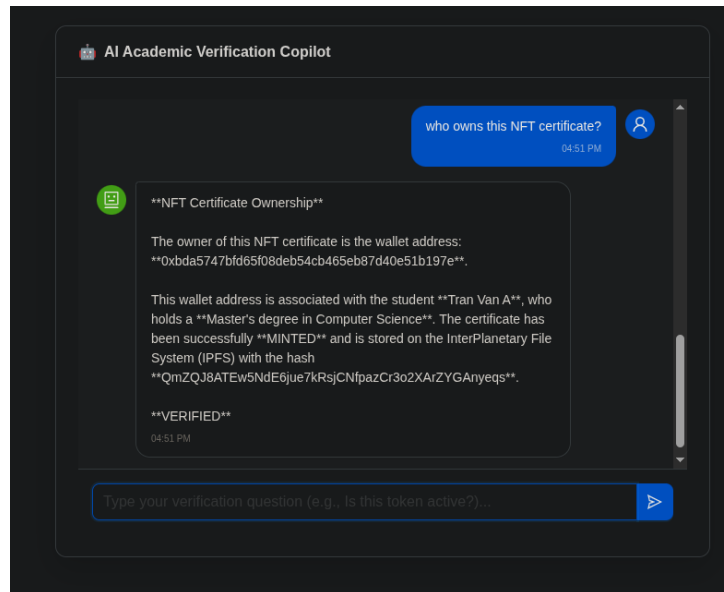


Figure 36: AI Assistant - Question: *Who own this NFT certificate?*

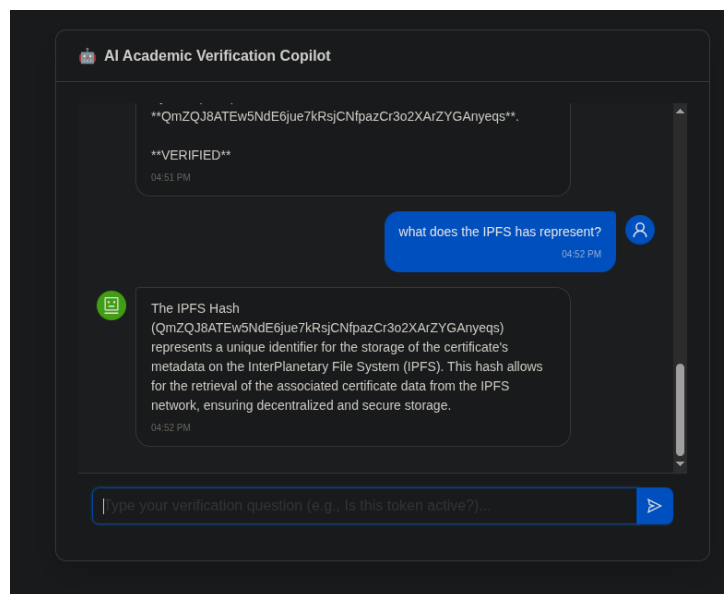


Figure 37: AI Assistant - Question: *What does the IPFS has represent?*

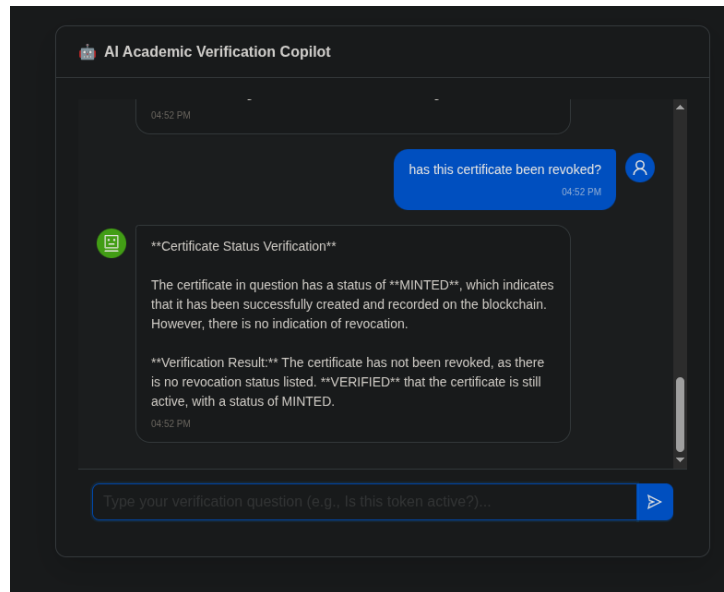


Figure 38: AI Assistant - Question: *Has this certificate been revoke?*

5.3.3 Step 3 – Context Construction

The backend service constructs a structured context block containing verified certificate information.

Student Name: Huynh Thi Luu

Token ID: 15

Degree: Bachelor of Software Engineering

Major: Software Engineering

Wallet Address: 0xA1B2...

IPFS Hash: QmXXXX...

Status: MINTED

5.3.4 Step 4 – AI Processing

The structured context and user question are submitted to the Llama 3.3 model through the Groq API. A system prompt constrains the model's behavior and instructs it to:

- Explain verification results.
- Interpret blockchain records.
- Describe NFT ownership information.
- Explain IPFS storage references.
- Avoid exposing raw JSON data.
- Produce concise professional responses.

5.3.5 Step 5 – Response Streaming

The generated response is streamed back to the frontend interface incrementally. This creates a conversational user experience and reduces perceived response latency.

5.4 AI Service Implementation

The AI service is implemented as a dedicated backend module responsible for communicating with the Groq cloud platform.

The system first assembles verified certificate information into a structured context object. This context is then combined with the user's question and submitted to the language model.

```
const context = `
  Verified Certificate Information

  Student Name: ${certificateData.blockchain.studentName}

  Token ID: ${certificateData.blockchain.tokenId}

  Degree: ${certificateData.blockchain.degree}

  Major: ${certificateData.blockchain.major}

  Wallet: ${certificateData.database.studentWallet}

  IPFS Hash: ${certificateData.blockchain.ipfsHash}

  Status: ${certificateData.database.lifecycleStatus}
`;
```

The context is subsequently injected into the LLM request:

```
return await groq.chat.completions.create({
  model: "llama-3.3-70b-versatile",

  messages: [
    {
      role: "system",
      content: prompt,
    },
    {
      role: "user",
      content: `
        Question:
        ${question}

        Certificate Payload Context:
        ${context}
      `,
    },
  ],

  temperature: 0.1,
  stream: true,
});
```

The AI-generated response is streamed through the Express backend and rendered progressively within the React chat interface.

5.5 Prompt Engineering and Hallucination Mitigation

A common challenge in LLM-based systems is hallucination, where the model generates information not supported by available data.

To reduce this risk, the proposed system applies a context-grounding strategy. Instead of allowing the model to independently retrieve blockchain information, verified certificate records are first collected by the application and injected directly into the prompt. The system prompt defines the following operational constraints:

- Explain only the supplied certificate information.
- Do not fabricate certificate attributes.
- Do not invent ownership records.
- Do not generate unsupported verification results.
- Clearly indicate when information is unavailable.
- Present responses using professional language.

```
const prompt = `You are an NFT Academic Certificate Verification Assistant.
```

```
  Responsibilities:
```

- ```
 1. Explain verification results.
 2. Verify certificate authenticity.
 3. Explain blockchain records.
 4. Explain IPFS storage.
 5. Explain NFT ownership.
 6. Help users understand certificate status.
```

```
 Rules:
```

- ```
    - Never output raw JSON.
    - Use professional formatting.
    - If certificate is verified, clearly state VERIFIED.
    - If information is missing, say so.
    - Keep responses concise.`;
```

In addition, the model temperature is configured to 0.1, reducing response randomness and encouraging deterministic outputs.

Blockchain Records + Database Records



Verified Certificate Context



Prompt Construction



Llama 3.3 Model



Human-Readable Verification Explanation

The AI assistant does not verify certificates by itself. Instead, it explains verification results that have already been obtained from blockchain and database records.

6 RESULTS & CONCLUSION

6.1 Project Results

The proposed NFT-based academic certificate management platform was successfully implemented and tested within a local Ethereum development environment using Hardhat. The system integrates blockchain technology, IPFS decentralized storage, MongoDB persistence, and an AI-assisted verification interface into a unified workflow. The implemented platform successfully demonstrated the following functions:

Certificate Issuance	Authorized administrators can generate academic certificates for eligible students. During issuance, the system: • Generates a PDF certificate • Calculates a SHA-256 hash of the generated document • Uploads the certificate PDF to IPFS • Creates and uploads a certificate metadata to IPFS • Mints an NFT certificate through the smart contract. • Stores transaction information in MongoDB
----------------------	---

Certificate Verification	The verification module successfully retrieves certificate information from blockchain records and associated database entries. Users can verify: • Certificate ownership • Student information • Academic degree details • NFT token identifier • IPFS metadata references • Certificate lifecycle status
Tampering Detection	To evaluate document integrity protection, modified certificate files were tested against the original blockchain records. The SHA-256 hash generated from a modified certificate differed from the hash recorded during issuance. Since the hashes no longer matched, the certificate could not be validated against the stored blockchain information. This result demonstrates the effectiveness of cryptographic hashing in detecting unauthorized document modifications.
AI-Assisted Verification	The AI assistant successfully generated natural-language explanations based on verified certificate information supplied by the application. Users were able to ask questions such as: • Is this certificate authentic? • Who owns this NFT certificate? • What does the IPFS hash represent? • What is the current certificate status? The assistant translated technical blockchain information into user-friendly explanations, improving accessibility for non-technical users.

Table 9: Project Results

6.2 Project Advantages

The proposed system provides several advantages compared with traditional centralized certificate management approaches.

Enhanced Integrity Protection	Certificate information is anchored to blockchain records and linked to cryptographic hashes. Any modification to the certificate document results in a different hash value, enabling reliable tampering detection.
Decentralized Storage	Certificate files and metadata are stored on IPFS rather than solely within centralized institutional servers. This reduces dependence on a single storage provider and improves data availability.
Verifiable Ownership	NFT certificates are minted directly to student-controlled wallet addresses. Ownership can be independently verified through blockchain records without requiring manual confirmation from the issuing institution.
Automated Verification	The platform allows certificate validation through automated blockchain queries, reducing the time and administrative effort required for manual verification processes.
Improved User Accessibility	The AI assistant converts technical blockchain information into understandable natural language responses, making verification results easier to interpret for employers and recruiters.

Table 10: Project Advantages

6.3 Project Limitations

Despite its successful implementation, several limitations remain.

Development Network Environment	The current implementation operates within a Hardhat local blockchain environment. Although suitable for development and testing, deployment to a public blockchain network is required before production use.
Wallet Adoption Challenges	The system relies on Ethereum-compatible wallets such as MetaMask. Users unfamiliar with blockchain technologies may require additional guidance during wallet registration and management.
External Service Dependencies	The platform depends on external services such as IPFS pinning providers and AI model providers. Service interruptions may temporarily affect metadata accessibility or AI response generation.

Wallet Ownership Verification	The current prototype captures wallet addresses through MetaMask authorization, but does not perform cryptographic message-signature verification. Future implementations may incorporate challenge-response authentication to provide stronger proof of wallet ownership.
System Authorization	If we wanted to expand this project, we would need to add role authentication, but since the main purpose of the project is to research NFTs, we will not add authorization functionality.

Table 11: Project Limitations

6.4 Future Work

Several enhancements can be considered to improve the system and support production deployment.

Public Blockchain Deployment	Future versions may be deployed to public Ethereum-compatible networks such as Sepolia, Polygon, or Arbitrum to support real-world usage and broader accessibility.
Enhanced Wallet Authentication	A cryptographic challenge-response mechanism could be implemented to verify ownership of submitted wallet addresses before linking them to student accounts.
Advanced AI Verification Assistance	Future AI modules could incorporate document analysis capabilities, allowing the system to inspect uploaded certificate files and provide additional verification insights.
Mobile Wallet Integration	Support for Wallet Connect and mobile wallet applications could improve accessibility for users who prefer smartphone-based blockchain interactions.
Academic Interoperability	Future development may support integration with multiple educational institutions, enabling cross-university issuance and verification of academic credentials.

Table 12: Project Future Works

6.4.1 Blockchain-Based Academic Transcript Verification

The current system focuses primarily on issuing and verifying NFT-based academic certificates. A potential extension of the platform is the incorporation of academic transcript verification. Instead of verifying only graduation certificates, the system could issue blockchain-backed transcript records containing detailed academic performance information, such as:

- Individual course results
- Semester GPA
- Cumulative GPA
- Academic honors and distinctions
- Course completion history

Similar to the certificate issuance process, transcript data could be hashed using SHA-256 and linked to decentralized storage through IPFS. A corresponding NFT or blockchain credential record could then be generated to represent the transcript. This approach would allow employers, universities, scholarship organizations, and professional certification bodies to independently verify academic achievements while reducing the risk of transcript forgery or unauthorized modifications.

Future research may also investigate privacy-preserving approaches, such as selective disclosure and zero-knowledge proof mechanisms, allowing students to reveal only specific academic information when required.

6.4.2 Institution-Specific Large Language Model Development

The current implementation utilizes a third-party cloud-hosted Large Language Model (Llama 3.3) through the Groq API. While this approach provides high-quality responses and rapid deployment, it introduces dependence on external AI service providers. A future enhancement would involve developing and deploying a dedicated institutional AI model trained on domain-specific educational data and certificate verification workflows.

Potential advantages include:

- Greater control over model behavior and outputs
- Reduced dependence on third-party AI providers
- Enhanced privacy and data governance
- Improved accuracy for academic credential verification tasks
- Custom knowledge integration for institutional regulations and policies

Future implementations may adopt open-source foundation models such as Llama, Mistral, or Qwen and fine-tune them using educational datasets, blockchain verification records, and institutional documentation. By hosting the model within the university infrastructure, all verification requests and certificate-related data could remain under institutional control, improving both privacy protection and operational independence.

6.5 Source Code Repository

The complete source code of the NFT Academic Certificate Verification System and the AI-assisted verification agent is available at:

- **GitHub Repository:** <https://github.com/khuyenabc123/blockchain>
- **Branch:** main
- **Guide:** README.md

The repository contains:

- Folders:
 1. backend: implement APIs, services, .etc
 2. frontend: implement UI
 3. blockchain: implement contract, store solidity file
- AI verification agent implementation
- Deployment configuration
- Documentation and setup instructions (README.md)

6.6 Conclusion

This research successfully designed and implemented a blockchain-based academic certificate management platform that combines NFT technology, IPFS decentralized storage, smart contracts, and artificial intelligence assistance.

The proposed system demonstrates how blockchain technology can be applied to academic credential management to improve transparency, integrity, and verification efficiency. By storing certificate metadata on IPFS and recording ownership information on blockchain networks, the platform provides a verifiable and tamper-resistant mechanism for managing academic records. In addition, the integration of an AI-assisted verification interface improves usability by translating technical blockchain information into understandable natural-language explanations.

Overall, the results indicate that combining Web2 technologies (React, Node.js, Express, and MongoDB) with Web3 infrastructure (Ethereum smart contracts, NFTs, and IPFS) provides a practical foundation for next-generation academic credential verification systems.

REFERENCES

- [1] Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, “An overview of blockchain technology: Architecture, consensus, and future trends,” in *Proceedings of the 2017 IEEE International Congress on Big Data (BigData Congress)*, 2017, pp. 557–564.
- [2] D. Yuan, Y. Luo, X. Zhuang, G. R. Rodrigues, X. Zhao, Y. Zhang, P. U. Jain, and M. Stumm, “Simple testing can prevent most critical failures: An analysis of production failures in distributed data-intensive systems,” *OSDI*, pp. 249–265, 2014.
- [3] I. Stoica, R. Morris, D. Liben-Nowell, D. R. Karger, M. F. Kaashoek, F. Dabek, and H. Balakrishnan, “Chord: A scalable peer-to-peer lookup protocol for internet applications,” *IEEE/ACM Transactions on Networking*, vol. 11, no. 1, pp. 17–32, 2003.
- [4] D. Yaga, P. Mell, N. Roby, and K. Scarfone, “Blockchain technology overview,” NIST, Tech. Rep., 2018. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/ir/2018/nist.ir.8202.pdf>
- [5] Wikipedia Contributors, “Bitcoin,” 2026. [Online]. Available: <https://en.wikipedia.org/wiki/Bitcoin>
- [6] L. Morel, “The Bitcoin white paper explained simply - chapter 1,” 2023. [Online]. Available: <https://www.bitstack-app.com/en/learn-bitcoin/the-bitcoin-white-paper-simply-explained---chapter-1>
- [7] C. T. Nguyen, D. T. Hoang, D. N. Nguyen, D. Niyato, H. T. Nguyen, and E. Dutkiewicz, “Proof-of-stake consensus mechanisms for future blockchain networks: Fundamentals, applications and opportunities,” *IEEE Access*, vol. 7, pp. 140 006–140 029, 2019.
- [8] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [9] V. Buterin and V. Griffith, “Casper the friendly finality gadget,” *arXiv preprint arXiv:1710.09437*, 2019. [Online]. Available: <https://arxiv.org/pdf/1710.09437>
- [10] N. Szabo, “Formalizing and securing relationships on public networks,” 1997.
- [11] M. I. Mehar, C. Shier, A. Giambattista, E. Gong, G. Fletcher, R. Sanayhie, H. M. Kim, and M. Laskowski, “Understanding a revolutionary and flawed grand experiment in blockchain: The dao attack,” Tech. Rep., 2017.
- [12] V. Buterin, “Ethereum: A next-generation smart contract and decentralized application platform,” 2014.
- [13] D. Perez and B. Livshits, “Smart contract vulnerabilities: Vulnerable does not imply exploited,” *USENIX Security*, pp. 2565–2582, 2020.
- [14] Solidity Team, “Solidity documentation v0.8.20,” 2026. [Online]. Available: <https://docs.soliditylang.org/>
- [15] G. Wood, “ETHEREUM: A secure decentralised generalised transaction ledger - Shanghai version efc5f9a,” *Ethereum Foundation Technical Specification*, 2025. [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>

-
- [16] W. Entriken, D. Shirley, J. Evans, and N. Sachs, “ERC-721: Non-fungible token standard specification,” *Ethereum Improvement Proposals (EIPs)*, no. 721, 2018. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-721>
- [17] A. Farabi, I. Khandaker, J. Ahsan, I. K. Shanto, N. Jahan, and M. J. Khan, “ShikkhaChain: A Blockchain-powered academic credential verification system for Bangladesh,” *arXiv preprint arXiv:2508.05334*, 2025. [Online]. Available: <https://arxiv.org/pdf/2508.05334>
- [18] Ethereum Team, “ERC-721 non-fungible token standard,” 2018. [Online]. Available: <https://ethereum.org/developers/docs/standards/tokens/erc-721/>
- [19] E. G. Weyl, P. Ohlhaber, and V. Buterin, “Decentralized society: Finding Web3’s soul,” *SSRN Electronic Journal*, 2022. [Online]. Available: <https://www.researchhub.com/paper/1270814/decentralized-society-finding-web3s-soul>
- [20] OpenZeppelin Team, “Smart contract ERC721 utilities and implementations,” 2026. [Online]. Available: <https://docs.openzeppelin.com/contracts/5.x/api/token/erc721>
- [21] J. Benet, “IPFS - content addressed, versioned, P2P file system,” *arXiv preprint arXiv:1407.3561*, 2014. [Online]. Available: <https://arxiv.org/pdf/1407.3561>
- [22] National Institute of Standards and Technology (NIST), “Secure hash standard (SHS),” *Federal Information Processing Standards Publication (FIPS) 180-4*, 2015. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>
- [23] World Wide Web Consortium (W3C), “Web cryptography API level 2: W3C recommendation,” 2025. [Online]. Available: <https://www.w3.org/TR/webcrypto-2/>
- [24] Ethers.js Contributors, “Ethers.js documentation v6,” 2026. [Online]. Available: <https://docs.ethers.org/v6/>
- [25] A. Grech and A. F. Camilleri, “Blockchain in education,” Joint Research Centre (JRC) Science for Policy Report, European Commission, Tech. Rep., 2017. [Online]. Available: https://etico.iiep.unesco.org/sites/default/files/2021-04/full_text_0.pdf
- [26] W. Gräther, S. Kolvenbach, R. Ruland, J. Schütte, C. F. Torres, and F. Wendland, “Blockchain for education: Lifelong learning passport,” Fraunhofer FIT Research Report, Tech. Rep., 2017. [Online]. Available: https://etico.iiep.unesco.org/sites/default/files/2021-04/full_text_0.pdf
- [27] K. Kumutha and S. Jayalakshmi, “The impact of the Blockchain on academic certificate verification system-review,” *EAI Endorsed Transactions on Energy Web*, vol. 8, no. 31, 2021.
- [28] MIT Media Lab, “Blockcerts,” *MIT Media Lab Project Report*, 2016. [Online]. Available: <https://www.blockcerts.org/>
- [29] M. Turkanović, M. Hölbl, K. Kočič, M. Heričko, and A. Kamišalić, “EduCTX: A Blockchain-based higher education credit platform,” *IEEE Access*, vol. 6, pp. 5117–5131, 2018. [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8247166>

-
- [30] X. Zhao and Y.-W. Si, “NFTCert: NFT-based certificates with online payment gateway,” *arXiv preprint arXiv:2202.09511*, 2022. [Online]. Available: <https://arxiv.org/pdf/2202.09511>
- [31] T. Rahman, S. I. Mouno, A. M. Raatul, A. K. A. Azad, and N. Mansoor, “Verifi-Chain: A credentials verifier using Blockchain and IPFS,” *arXiv preprint arXiv:2307.05797*, 2023. [Online]. Available: <https://arxiv.org/pdf/2307.05797>
- [32] P. Khati, A. K. Shrestha, and J. Vassileva, “Student certificate sharing system using Blockchain and NFTs,” *arXiv preprint arXiv:2310.20036*, 2023. [Online]. Available: <https://arxiv.org/pdf/2310.20036>
- [33] R. Priyadarshini, R. Pandey, K. C. Ankit, D. Bhandari, B. Khadka, R. K. Barik, and M. J. Saikia, “A faster, integrated, and trusted certificate authentication and issuer validation system based on Blockchain,” *IEEE Xplore*, vol. 13, pp. 14 210–14 225, 2025. [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10872957>
- [34] J. A. T. Fairfield, “Tokenized: The law of non-fungible tokens and unique digital property,” *Indiana Law Journal*, vol. 97, no. 4, pp. 1261–1313, 2022.
- [35] Hardhat Team, “Hardhat: Ethereum development environment,” 2026. [Online]. Available: <https://hardhat.org/docs/getting-started>
- [36] Q. Wang, R. Li, Q. Wang, and S. Chen, “Non-fungible token (NFT): Overview, evaluation, opportunities and challenges,” *arXiv preprint arXiv:2105.07447*, 2021. [Online]. Available: <https://arxiv.org/pdf/2105.07447>
- [37] Ethereum Team, “Gas and fees,” 2025. [Online]. Available: <https://ethereum.org/developers/docs/gas/>